

Fast Candidate Generation for Support-Minimizing Build Orientation in 3D Printing Using a Support Flow Tensor Flow

InHwan Sul

Abstract

Build orientation strongly affects support volume, material use, printing time, and post-processing quality in additive manufacturing. Accurate support-volume evaluators, including the authors' previous GPU-based modified support-structure tomography (GPU-MSST; here denoted TOMO_INT3), can search a yaw-pitch grid, but they still evaluate every sampled direction and therefore retain a large fixed search cost. This paper proposes a fast candidate generator, the Support Flow Tensor Field (SFTF), whose role is not to replace an accurate slicer or TOMO evaluator, but to provide a small set of promising build-orientation candidates for subsequent local verification.

For each sampled build direction, SFTF estimates support tendency from two geometric cues: whether an overhang face can be supported by another visible face beneath it, and how much support must extend directly to the build plate. The first cue is obtained by ray-cast face-to-face visibility and is accumulated as an asymmetric support-flow tensor; the second is a build-plate penalty. The build-plate term can also be written as the Rayleigh contribution of a virtual ground receiver added to the tensor, so the main support cue remains within one augmented tensor formulation. The derivation is given in the Appendix, while the main text emphasizes the resulting workflow: coarse spherical sampling, local resampling around good candidates, angular non-maximum suppression to report separated orientation basins, and final verification with TOMO or a slicer.

On stored TOMO_INT3 grids, cross-validated SFTF candidates achieved a best-of-three support-volume ratio of 2.37, compared with 3.63 for PCA axes and 4.30 for symmetric Support Tensor eigenvectors. On a controlled five-group shape set ranging from primitives to meshes with about 10^6 faces, Python SFTF reduced computation time by 1.4–34 $\times$ relative to a 3 $^\circ$ TOMO_INT3 exhaustive sweep, and the C++ implementation by 10–5,160 $\times$. The experiments also show that changing the exhaustive-sweep backend from INT3 to CUDA is not always faster on real and large meshes, whereas reducing the number of directions is consistently beneficial. The method is therefore best interpreted as a fast, interpretable warm-start for support-minimizing build-orientation search.

Keywords: additive manufacturing, build orientation optimization, support minimization, candidate generation, support-flow tensor field, 3D printing

1 Introduction

1.1 Build Orientation and the Support Structure Problem

In additive manufacturing, and especially in the most widely used FDM (fused deposition modeling) method, molten filament is stacked layer by layer [8]. To print a downward overhang steeper than the critical angle (critical angle, θ_c), a support structure (support structure, SS) is needed to hold it up from below. The support structure is a sacrificial structure that is removed after printing, but it consumes additional filament and print time, and degrades surface quality through removal marks and staircasing. Therefore the support volume is a key factor governing the cost and quality of FDM printing.

The scope of this paper is the standard single-build-direction setting in which support demand is mainly determined by downward overhangs and a printable critical angle. This setting

covers common slicer-style support estimation for extrusion-based 3D printing and also provides a geometric benchmark for support-volume evaluators such as MSST/TOMO. It does not attempt to model process-specific thermal distortion, residual stress, or multi-axis deposition; these effects can be added later as additional objectives after promising orientation candidates have been generated.

Even for the same shape, the distribution of overhangs changes depending on the pose in which it is placed on the build plate, so the build orientation directly affects the following factors.

- Support volume
- Print time
- Material usage
- Surface quality
- Print stability

Among these, finding the build orientation that minimizes the support volume is the optimal-orientation search problem that this study addresses.

1.2 Related Work

Build orientation optimization and support structure reduction are long-studied topics in additive manufacturing design, and prior approaches can be broadly divided into (i) *orientation optimization*, which finds a build orientation with a small support volume, (ii) *support structure design*, which optimizes the shape of the support structure for a given orientation, and (iii) *design-stage approaches*, which reduce the support requirement itself by changing the geometry or topology of the part.

In orientation optimization, Das et al. voxelized the mesh and selected a build orientation that jointly considers the support volume and part error [4], and Wang et al. used a layered depth-normal image representation and GPU acceleration to quickly process the geometric operations of complex polyhedra, laying the foundation for voxel-based additive manufacturing computation [30]. Such explicit, voxel-based evaluation gives the support volume of a single direction accurately, but when searching over directions exhaustively it incurs a large fixed cost independent of shape complexity (shown quantitatively in §13).

On the support structure design side, research that minimizes support volume with lattice or cellular structures is active. Vaissier et al. pruned lattice support structures with a genetic algorithm to reduce material consumption [29], and Jiang et al. experimentally identified the printable critical overhang angle in extrusion-based additive manufacturing to reduce unnecessary support [14]. As attempts to reduce the support requirement at the design stage, Mirzendehtdel and Suresh proposed topology optimization that includes support structures as a constraint [21], and Langelaar presented a unified formulation that simultaneously optimizes part topology, support placement, and build orientation [19]. Meanwhile, Dai et al. demonstrated an alternative paradigm that achieves support-free printing using multi-axis motion [3].

In particular, a line of work that expresses the directional dependence of overhang directly as *anisotropy* or a tensor has developed within the topology-optimization community, and this is the family most closely related to the tensor formulation of the present study. Gaynor et al. introduced a maximum overhang constraint into SIMP topology optimization, forcing each layer to be supported by the solid below it within the self-supporting angle [7]. Garcke et al. penalized overhanging boundaries through anisotropic interface energy (an anisotropic interface energy of Wulff-shape type) in a phase-field topology optimization, so that self-supportable boundary-normal directions are preferred [6]. Shin et al. targeted multi-axis additive manufacturing and simultaneously optimized a deposition-direction-dependent material stiffness anisotropy tensor

$\mathbf{D}(\phi) = \mathbf{T}(\phi) \mathbf{D}_0 \mathbf{T}(\phi)^T$, an overhang penalty, and the stage-wise build orientations [25]. On the industrial side, Smith and Hollenbeck patented a method that selects the build orientation of complex metal parts with many orthogonal and internal faces—where support is hard to remove—so as to minimize support structure and surface distortion [26].

This study also shares with the anisotropy family above the idea of capturing the directionality of overhang with a tensor, but its position is complementary. The tensors in those works represent the *physical* anisotropy of material stiffness (Shin) or interface energy (Garcke) *inside* a PDE-constrained design optimization that changes the part topology/density, with the build direction either fixed (Gaynor, Garcke) or co-optimized as a design variable (Shin). By contrast, the Support Flow Tensor Field of this study is a purely geometric support-flow descriptor evaluated per candidate direction on a *fixed* part, used to *rank* build orientations at low cost without redesigning the part. That is, without changing the shape of the support structure or the part topology, and assuming standard FDM single-direction printing, it focuses on the *build orientation*, but rather than replacing the expensive exhaustive evaluation, it aims for a candidate generator and warm-start that, in front of it, narrows down promising directions at low cost and passes them to an accurate verifier or the local verification of a commercial slicer. The patent of Smith and Hollenbeck [26] is close to this study in goal (selecting the build orientation of a fixed shape) but is an industrial method whose procedure is not disclosed, whereas this study provides a transparent tensor cost and a verifier warm-start. The next section first summarizes the implicit-estimation family that this study directly extends (shadow-volume-based MSST).

1.3 Optimal Orientation Search: Explicit/Implicit Methods and MSST

The optimal build orientation can be obtained explicitly or implicitly. Explicit methods actually slice each candidate direction to generate G-code and directly compute the support volume. They are accurate but expensive, so prior works, even after voxelizing the mesh and accelerating G-code generation with GPUs, generally remained at single-direction evaluation due to the computational burden and could not be applied to exhaustive optimal-orientation search.

Implicit methods indirectly estimate the support volume without complex G-code generation. Representatively, Ezair et al. observed that the volume of the support structure is similar to the shadow volume produced by a virtual sun [5]. This is intuitive but holds only when the critical angle is 0° , i.e., when every overhang requires support. The authors’ prior work [16] generalized this idea to an arbitrary critical angle and developed it into “modified support structure tomography (MSST).”¹

MSST divides the virtual ray into many rays and treats each ray’s intersection with the surface as a “pixel.” Within a “pixel slot” that sorts pixels at the same (x, y) location by their Z coordinate, it classifies upward-facing faces (α) and downward-facing faces (β), and estimates the support volume (V_{ss}) by subtracting the object volume (V_o) and the non-overhang volume (V_{nv}) from the top-cover volume (V_{tc}) in the form $V_{ss} = V_{tc} - V_o - V_{nv}$. This approach reproduced almost the same support volume as a commercial slicer on several standard meshes such as the Stanford Bunny.

1.4 GPU-MSST and Remaining Limitations

The only weakness of MSST was speed. A single-direction computation of the 65k-triangle Bunny takes about 1 second, and sweeping yaw–pitch at 1° intervals requires $360 \times 360 = 129,600$ single-direction computations, so a single optimal-orientation search takes 1–2 minutes on a CPU. The authors recently proposed GPU-MSST, which reconstructs MSST with GPU-friendly data structures (slot-based pixel data structure, triangle-split voxelization) [18], shortening the same exhaustive search to about 10 seconds and reporting on average $11.2\times$ acceleration relative to a

¹The authors’ prior MSST [16] and GPU-accelerated (GPU-MSST) [18] work. Public source code: https://github.com/cfms-lab/Tomo_GPU2024.

high-end CPU and up to $62\times$ relative to a low-cost CPU. In this paper we use this GPU-based MSST implementation as an accurate support-volume verifier and refer to it as TOMO_INT3.

However, GPU-MSST also essentially searches the entire yaw–pitch grid *exhaustively*. That is, the acceleration made single-direction evaluation fast; it did not reduce the number of directions to be evaluated (129,600). As a result, the search cost has a large fixed cost almost independent of shape complexity (in §13 of the main text, even a cube with only 12 faces takes about 4 seconds for a 3° TOMO_INT3 sweep), and is excessive for simple shapes or fast preliminary review. Indeed, the prior work itself started from the observation that “only a limited portion between the α faces and β faces contributes to the final support structure,” and presented, as future work, a tensorial approach (tentatively named shadow tensor) that reduces the evaluation targets. This study is precisely the follow-up work in that direction.

1.5 This Study: Support Flow Tensor Field

The goal of this study is not to *replace* the expensive exhaustive search, but to build a candidate generator that quickly suggests promising build orientations in front of it, serving simultaneously as a slicer/TOMO warm-start. By evaluating a low-cost support cost for each candidate direction on the sphere, it narrows down only a small number of promising orientations, and feeds these into the local verification of TOMO_INT3 or a commercial slicer, thereby reducing the overall search cost.

For this, a tensor that summarizes the shape is needed. The simplest candidate is the symmetric tensor that collects the distribution of face normals with area weighting, $M = \sum_i A_i m_i m_i^T$, which in this paper we call the “Support Tensor” as a comparison baseline. However, as shown in §2, this symmetric tensor only captures the *distribution* of normals and cannot explain where the support structure actually flows from and to. In particular, it does not reflect reentrant or internally occluded regions, the height to the build plate, or the visibility changes that depend on the build orientation.

Therefore, for direction $n \in S^2$ this study directly evaluates an *asymmetric* Support Flow Tensor Field $F(n)$ (§3). $F(n)$ accumulates, via ray casting, the face-to-face support “flow” from an overhang face toward the actual receiver face beneath it, and is treated not as a simple eigenvector problem but as a per-direction cost evaluation problem. Also, as the core unifying perspective of this study, the build-plate support cost is formulated not as a separate scalar penalty but as the Rayleigh contribution obtained when adding a “virtual ground node whose normal is the build direction” to the flow tensor. In this way, face-to-face self-support and build-plate support are unified into a single flow tensor (unified Rayleigh cost $\tilde{R} = R + B$, §5.1), and in the quantitative analysis (§12.2) we confirm that this unified term drives the predictive power.

In summary, the contributions of this study are as follows.

- We formulate build-orientation selection as a fast candidate-generation problem before accurate TOMO/slicer verification, rather than as a complete replacement for exhaustive support-volume search.
- We define an asymmetric Support Flow Tensor Field $F(n)$ for each build orientation, using ray casting to distinguish internal face-to-face self-support from support that must reach the build plate.
- We show that the build-plate support cost can be interpreted as the Rayleigh contribution of a virtual ground receiver added to the flow tensor (Eq. (16)); the derivation is kept in the Appendix so that the main text can focus on the AM workflow.
- Through correlation analysis, ablation, and cross-validation on a stored TOMO_INT3 grid, we identify which terms actually predict support volume and verify the advantage over PCA and symmetric Support Tensor eigenvector baselines.

- On a difficulty-controlled five-group shape set, including meshes with about 10^6 faces, we compare Python SFTF and C++ SFTF with TOMO_INT3/TOMO_CUDA 3° exhaustive sweeps and show that reducing the number of evaluated orientations gives a larger and more stable speed gain than changing the exhaustive-sweep backend alone.

For readability, the main text can be read as a four-step workflow: (1) detect direction-dependent overhangs, (2) estimate whether they can flow to another face or must reach the build plate, (3) rank a small candidate pool of build orientations, and (4) verify only those candidates with TOMO or a slicer. The algebraic proof that the build-plate term can be absorbed into an augmented tensor is provided in Appendix A.

2 The Baseline (Symmetric) Support Tensor and Its Limitations

Terminology (important). Since two tensors with similar names appear in this paper, we first distinguish them. The *Support Tensor* is a **symmetric** tensor that collects the distribution of face normals with area weighting; it is *not* newly proposed by this study but is a classical construction used as a comparison baseline. It belongs to the same family as the orientation tensor (or fabric tensor) of fiber/particle mechanics or the structure tensor of image processing, i.e., the “second moment of the normal vectors.”² By contrast, what this study newly proposes is the direction-dependent **asymmetric** tensor of the next section (§3), the *Support Flow Tensor Field* $F(n)$. In short, the “Support Tensor” is an existing baseline, and the “Support Flow Tensor Field” is the contribution of this study. Meanwhile, the anisotropic interface-energy or stiffness tensors used to handle overhang in topology optimization [6, 25] form yet a third category distinct from both: they are not descriptors of the directional cost of a fixed shape but *physical* anisotropy models within a design optimization that redesigns the part (§1.2).

Definition and Properties. The baseline (symmetric) Support Tensor is defined as

$$M = \sum_i A_i m_i m_i^T \quad (1)$$

Here A_i is the area of triangular face i , and m_i is its outward normal. Since M is a symmetric positive semidefinite matrix,³ an eigenvector⁴ interpretation is possible. That is, the eigenvectors of M tell us which direction the shape’s normals point on average (the principal direction).

Limitations. However, M only captures the *distribution* of normals and not the *process* by which the support structure is actually generated. Specifically, M does not include the following information.

- Which faces are overhangs in a particular direction
- Whether there is another actually supportable face beneath an overhang face
- The height or distance to the support target
- The region that the build plate must directly support
- The asymmetric face-to-face support flow

²The symmetric tensor $\sum_i A_i m_i m_i^T$ is the (area-weighted) outer-product sum of the normals m_i , a classical shape descriptor of the structure tensor / orientation tensor family. See: [17].

³A symmetric matrix such that $x^T M x \geq 0$ for all vectors x . This means all eigenvalues are ≥ 0 . See: [13]

⁴A direction x satisfying $Mx = \lambda x$, representing the axes that the matrix “stretches” most strongly. See: [9]

Therefore, the eigenvectors of the Support Tensor can explain the statistical directionality of the shape, but they do not always coincide with the optimal direction of the support volume that the slicer computes. This limitation is the starting point for introducing a per-direction asymmetric flow tensor in the next section.

3 Per-Direction Support Flow Tensor Field

We take the build orientation as a unit vector

$$n \in S^2, \quad \|n\| = 1 \quad (2)$$

Letting the centroid of face i be c_i , the outward normal m_i , and the area A_i , the overhang coefficient at direction n is defined as

$$O_i(n) = \max(0, -m_i \cdot n) \quad (3)$$

Let j be the first valid face encountered by casting a ray in direction $-n$ from overhang face i . The height difference is then

$$h_{ij}(n) = (c_i - c_j) \cdot n \quad (4)$$

and only the case $h_{ij} > 0$ is recognized as a support flow.

The face-to-face influence is set as

$$w_{ij}(n) = \frac{O_i(n)}{1 + \alpha h_{ij}(n)} \quad (5)$$

The current implementation uses

$$\alpha = \frac{1}{D} \quad (6)$$

where D is the bounding box diagonal of the input mesh.

Therefore the Support Flow Tensor for direction n is computed as

$$F(n) = \sum_{(i,j) \in \mathcal{R}(n)} w_{ij}(n) A_i A_j m_i m_j^T \quad (7)$$

Here $\mathcal{R}(n)$ is the set of valid face-to-face support relationships obtained by ray casting.⁵ Since $F(n)$ is a sum of $m_i m_j^T$, it is in general not a symmetric matrix.

4 Ray Casting and Valid Support Relationships

For each candidate direction n , ray casting every overhang face would be computationally expensive. Therefore, the current implementation selects source faces by deterministic stratified sampling⁶ in proportion to the overhang contribution

$$A_i O_i(n) \quad (8)$$

The number of rays is set according to the number of faces N_f as

$$K_{ray} = \text{clip}(0.02N_f, 500, 3000) \quad (9)$$

⁵A technique that shoots a ray to find the first surface it encounters. Here it is used to shoot a ray from an overhang face in the direction opposite the build direction ($-n$) to find the receiver face below. See: [23]

⁶Stratified sampling. A method that divides the whole into several intervals (strata) and then draws samples from each stratum, obtaining a more evenly representative sample than simple random sampling (stratified sampling). See: [2]

To avoid self-intersection, the ray origin is slightly shifted as

$$r_i(0) = c_i - \varepsilon n \quad (10)$$

and the ray direction is $-n$. The current implementation uses

$$\varepsilon = 10^{-6} D \quad (11)$$

A valid receiver face j must satisfy the following conditions.

1. $j \neq i$.
2. The ray distance is sufficiently positive.
3. $h_{ij}(n) > 10^{-5} D$.
4. The receiver normal faces the build direction sufficiently.

The last condition is implemented as

$$m_j \cdot n > \eta \quad (12)$$

and currently uses $\eta = 0.05$. This condition removes vertical or flipped faces that are hard to regard as actual receiver faces even if the ray reached them.

5 Build Plate Term

An overhang face for which ray casting cannot find a support face below must eventually be held up by the build plate or by a separate support structure. The current implementation computes this term as a scalar, but as shown in §5.1, it is exactly equal to the Rayleigh contribution obtained when adding the build plate to the flow tensor as a virtual receiver.

The build-plate height reference for direction n is defined as

$$z_{plate}(n) = \min_{v \in V} v \cdot n \quad (13)$$

The height of face i to the build plate is

$$h_{iP}(n) = c_i \cdot n - z_{plate}(n) \quad (14)$$

Letting $\mathcal{B}(n)$ be the set of source faces with no valid face-to-face hit, the build-plate support penalty is computed as

$$B(n) = \sum_{i \in \mathcal{B}(n)} O_i(n) A_i (1 + \alpha h_{iP}(n)) \quad (15)$$

This term increases the cost the more a tall overhang directly requires support all the way to the build plate.

5.1 Virtual Ground Node Interpretation of the Build Plate

$B(n)$ looks like a scalar penalty added outside the tensor, but it is in fact equivalent to the Rayleigh contribution obtained when adding the build plate as a virtual receiver (ground node) to the face-to-face flow tensor $F(n)$. The idea is that the support-face normal of the build plate is exactly the build direction n ($m \cdot n = 1$, i.e., the limiting receiver that faces most upward), so a face $i \in \mathcal{B}(n)$ supported by the plate can be treated as a “flow pair with a virtual node whose

normal is n .” Adding these virtual pairs to the face-to-face flow tensor yields an *augmented flow tensor* $\tilde{F}(n)$, whose symmetric part $\tilde{F}_s(n)$ satisfies the exact identity

$$\tilde{R}(n) = \max(0, -n^T \tilde{F}_s(n) n) = R(n) + B(n) \quad (16)$$

That is, the unified Rayleigh cost $\tilde{R}$ equals the sum of the face-to-face Rayleigh term R and the build-plate penalty B . The explicit construction of $\tilde{F}(n)$ and the term-by-term derivation of Eq. (16) are given in Appendix A. This equivalence was also numerically confirmed as $\max |\tilde{R} - (R+B)| < 10^{-12}$ over the entire candidate pool of the five stored meshes (about 1,292 candidates each) (§12.2).

The significance of this reformulation is twofold. First, the build-plate cost that accounts for most of the predictive power (§12.1) turns out to be not an ad-hoc scalar but a term of the flow tensor, justifying the “tensor” formulation. Second, the build plate is not a separate mechanism but is unified as “the limit of face-to-face self-support (a fully horizontal receiver)” — every overhang flows to a receiver, and that receiver is either another face or the build plate. The asymmetry of the height convention is intentional. The face-to-face weight uses attenuation $1/(1+\alpha h)$ (distant self-support is weakened), while the plate term uses amplification $(1+\alpha h)$ (a tall support column is costly). That this sign difference is not arbitrary is verified by quantitative comparison with other conventions in §12.2.

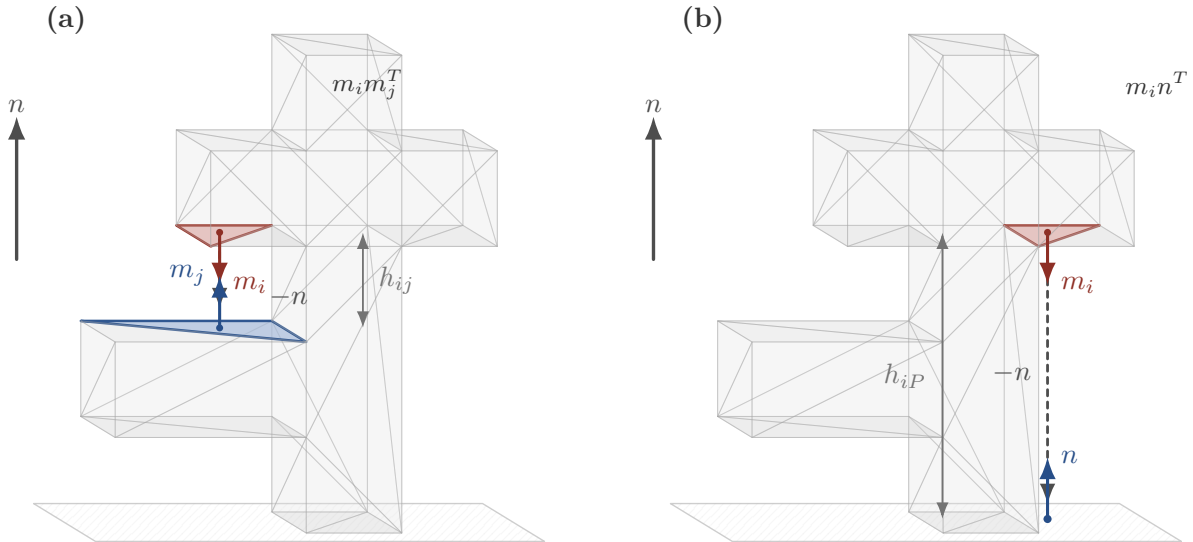


Figure 1: Unified support-flow interpretation on the nonconvex FTree4x example. The semi-transparent mesh is rotated 180° about the build direction n for visibility. (a) Face-to-face flow: an overhang face i casts a ray along $-n$ and is supported by a real receiver face j , contributing $m_i m_j^T$ with support height h_{ij} . (b) Build-plate flow: when no valid receiver face exists below the overhang, the build plate is treated as a virtual receiver whose normal is n , contributing $m_i n^T$ with support height h_{iP} .

6 Per-Direction Objective Function

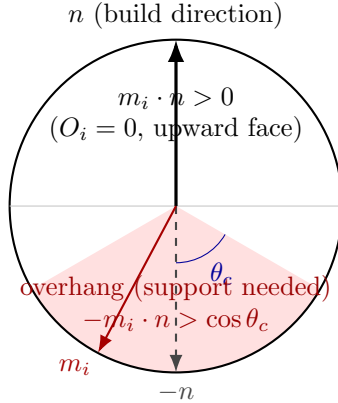


Figure 2: Geometric meaning of the overhang coefficient $O_i(n) = \max(0, -m_i \cdot n)$. The possible face-normal directions are shown on a unit circle with respect to the build direction n . The upper semicircle ($m_i \cdot n > 0$) consists of upward faces with $O_i = 0$, and the lower shaded region is the cone of half-angle θ_c around $-n$, where faces with $-m_i \cdot n > \cos \theta_c$ are overhangs that actually require support. O_i grows the more the normal aligns with $-n$ (the further down it goes).

To use the asymmetric tensor $F(n)$ as a directional cost, we define the symmetric component as

$$F_s(n) = \frac{F(n) + F(n)^T}{2} \quad (17)$$

The initial approach used $n^T F_s(n) n$ directly, but experiments showed that under strong support flow the sign of the value could become negative, flipping the alignment direction. Therefore the current implementation converts the Rayleigh term⁷ into a non-negative support cost.

$$R(n) = \max(0, -n^T F_s(n) n) \quad (18)$$

The face-to-face support volume term is set as

$$P(n) = \sum_{(i,j) \in \mathcal{R}(n)} \left(\frac{O_i(n)}{1 + \alpha h_{ij}(n)} A_i A_j \right) (1 + \alpha h_{ij}(n)) \quad (19)$$

Here the height attenuation factor $1/(1 + \alpha h_{ij})$ included in the face-to-face influence w_{ij} and the outer $(1 + \alpha h_{ij})$ cancel each other, so $P(n)$ ultimately simplifies to

$$P(n) = \sum_{(i,j) \in \mathcal{R}(n)} O_i(n) A_i A_j \quad (20)$$

That is, $P(n)$ represents the area-weighted overhang amount itself over the valid hits, and the height dependence is intentionally removed. This is a choice to keep the height attenuation in the tensor $F(n)$ so as to weaken the contribution of distant face-to-face flows, while interpreting the face-to-face support volume term $P(n)$ as the pure self-supportable area amount.

⁷Rayleigh quotient: for a symmetric matrix A and vector x , the scalar defined as $\frac{x^T A x}{x^T x}$. Since $\|n\| = 1$ here, $n^T F_s(n) n$ itself is the Rayleigh quotient at direction n , and we flip its sign to interpret it as a “support cost.” (A quantity closely related to eigenvalues; [13])

Using the augmented tensor of §5.1, the Rayleigh term R and the build-plate term B merge into a single unified Rayleigh cost $\tilde{R} = R + B$ (Eq. (16)). Therefore the final objective function can be written as the sum of the unified tensor term and the face-to-face support volume

$$J(n) = \lambda_R R(n) + \lambda_P P(n) + \lambda_B B(n) = \underbrace{\tilde{R}(n)}_{R+B} + P(n) \quad (21)$$

with ($\lambda_R = \lambda_P = \lambda_B = 1$), and from the unified-tensor perspective it is interpreted as “the Rayleigh support cost $\tilde{R}$ of the augmented flow tensor plus the face-to-face support area amount P .” As shown in §12.2, $\tilde{R}$ itself is a better predictor than J , because the summed P acts as a negative signal. In the initial implementation, from the singular values⁸ $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$ of $F(n)$ we added a nuclear score⁹ $S(n) = \sum_k \sigma_k(n)$ and a $\sigma_1(n)$ term with weight 0.05, but in the ablation of §12.1 these two terms were confirmed not to change the candidate selection at all, so they were excluded from the objective function. The singular values are kept only as auxiliary features for post-processing rank tuning.¹⁰ (§12.3). The optimal build orientation is defined as

$$n^* = \arg \min_{n \in S^2} J(n) \quad (22)$$

7 Spherical Sampling and Local Re-search

Using only three eigenvectors does not sufficiently reflect the asymmetry and direction dependence of the Support Flow Tensor Field. Therefore the current implementation uses the following two-stage search that follows a simple coarse-to-fine strategy of evaluating a sparse global sample and then densely resampling only around promising candidates.

1. Generate 512 coarse directions by the Fibonacci sphere [24, 10] method.¹¹
2. Compute $J(n)$ for each direction.
3. Select the top 12 candidates, but remove directions that are too close to each other.
4. Resample around each top candidate with 2° , 4° , 6° , 8° cones.
5. For each cone angle, evaluate 16 azimuths.
6. Combine the coarse and refined candidates and re-sort.
7. Finally return the top 3 directions that are at least 3° apart from each other.

This approach is faster than densely evaluating the entire sphere, while still being able to reflect sensitivity to small angular changes around the optimal direction.

⁸Singular value and singular value decomposition (SVD): the non-negative scale factors that appear when decomposing an arbitrary matrix into rotation–scale–rotation, representing how much the matrix “stretches” along each axis. For the non-symmetric $F(n)$, singular values are used instead of eigenvalues. See: [9]

⁹Nuclear norm: the sum of singular values $\sum_k \sigma_k$. A type of Schatten norm, a measure of the overall “size” of a matrix (a type of matrix norm). See: [13]

¹⁰In this paper, “rank tuning” is not a standard term but refers to the post-processing procedure of this study. Each feature is normalized to its rank within the candidate set ($0 \sim 1$) (see the formula in §12.3), and then linear weights are learned on these rank-normalized features to re-rank the candidates. For the concepts of rank and rank correlation, see [27].

¹¹A method that scatters points nearly uniformly on a sphere using the golden angle ($\approx 137.5^\circ$), commonly used to obtain evenly distributed direction samples (the 137.508° rotation in the code is this). For classical results on quasi-uniform distributions of points on a sphere, see Saff and Kuijlaars [24]; for the spherical application of the Fibonacci lattice and its uniformity analysis, see González [10].

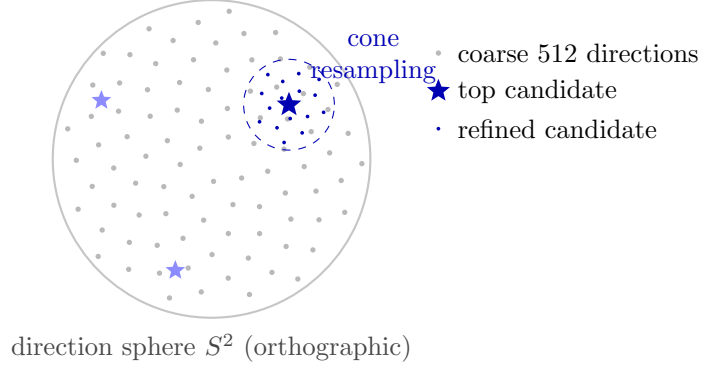


Figure 3: Two-stage direction search. After evaluating coarse candidates (gray, 512) that sparsely cover the sphere via the Fibonacci sphere, the regions around the top candidates (stars) are densely resampled with 2° – 8° cones (blue dots) to reflect sensitivity near the optimal direction. Local precision is obtained without densely evaluating the entire sphere.

8 Computational Acceleration and Final Candidate Rescoring

Since the per-direction Support Flow Tensor Field casts many rays for each candidate direction, a direct implementation has its computational bottleneck mostly in ray–triangle intersection [22]. The initial implementation obtained all hits for each ray and then sorted by distance to find the first valid receiver face. However, since for most rays the nearest hit is immediately the valid hit, computing all hits every time creates unnecessary cost.

The current implementation first performs a first-hit query with

$$\text{multiple_hits} = \text{False} \quad (23)$$

Only when this first hit violates the self-face, too-short distance, negative height difference, or receiver normal condition is the existing multiple-hit query performed as a fallback for that ray. That is, over the entire ray set $\mathcal{Q}$, only

$$\mathcal{Q}_{\text{fallback}} = \{q \in \mathcal{Q} \mid \text{firstHit}(q) \text{ is invalid}\} \quad (24)$$

is searched additionally. This strategy maintains the existing exact valid-hit selection rule, while reducing the number of intersection candidates and the sorting cost for most rays.

Also, the face normals, areas, centroids, vertex arrays, bounding box diagonal D , $\alpha = 1/D$, ray offset ε , and height tolerance, which used to be recomputed for each candidate direction, were computed only once per mesh and cached. For this, the current implementation shares

$$\mathcal{D}_{\text{mesh}} = \{m_i, A_i, c_i, V, D, \alpha, \varepsilon, \varepsilon_h\} \quad (25)$$

across the entire Support Flow evaluation.

In profiling on the Bunny 1k mesh, the existing direction evaluation time was about 15.7 seconds, but after applying first-hit fallback and mesh surface data caching it decreased to about 1.5 seconds. On the same mesh, the selected top direction, score, and hit count matched those of the existing multiple-hit method. Therefore this optimization can be viewed as an implementation-level acceleration that reduces only the computational amount without changing the meaning of the algorithm.

8.1 Final Candidate Rescoring Based on TOMO Results

The original objective function $J(n)$ was effective for candidate generation and local re-search, but it was not always well aligned with the actual support volume v_{ss} of TOMO_INT3 when

choosing the final top 3 candidates. In particular, a direction with a small $B(n)$ term is not always a direction with a small actual support volume, and a candidate with too small a face-to-face flow amount and hit count may not actually form sufficient self-support relationships.

Therefore, after the coarse and refined candidate pool has been generated, the current implementation applies a final rank-based rescoring step. The rescoring uses the intra-pool ranks of J , R , P , B , hit count, and singular-value descriptors, rather than their raw numerical values. This keeps the candidate generator itself simple and geometric, while allowing the final ordering to incorporate empirical feedback from TOMO or a slicer. The detailed formula and the calibration weights used for the reported in-sample values are given in Appendix B.

Two points are important for interpreting the results. First, the calibrated weights are not claimed to be universal physical constants; they are an example of using accurate support-volume feedback to rank an already small candidate pool. Second, the paper reports Leave-One-Mesh-Out cross-validation in §12.3, so the reader can distinguish the optimistic in-sample ranking from a more conservative generalization estimate.

9 Algorithm Overview

Algorithm 1 Augmented (ground-node) Support Flow Tensor Evaluation with Final Rank Tuning

```

1: for each sampled build direction  $n$  do
2:   Normalize  $n$ 
3:   Compute  $O_i(n) = \max(0, -m_i \cdot n)$  for all faces
4:   Select  $K_{ray}$  source faces by stratified sampling with weight  $A_i O_i(n)$ 
5:   Initialize  $\tilde{F} \leftarrow 0$   $\triangleright$  augmented flow tensor (face-to-face + virtual ground node),  $P \leftarrow 0$ 
6:   for each sampled source face  $i$  do
7:     Cast ray from  $c_i - \varepsilon n$  along  $-n$ 
8:     Find the first valid receiver face  $j$ 
9:     if valid  $j$  exists then
10:       $h_{ij} \leftarrow (c_i - c_j) \cdot n$ ;  $w_{ij} \leftarrow O_i(n)/(1 + \alpha h_{ij})$ 
11:       $\tilde{F} \leftarrow \tilde{F} + w_{ij} A_i A_j m_i m_j^T$   $\triangleright$  face-to-face flow
12:       $P \leftarrow P + w_{ij} A_i A_j (1 + \alpha h_{ij})$ 
13:    else
14:       $h_{iP} \leftarrow c_i \cdot n - z_{plate}(n)$ 
15:       $\tilde{F} \leftarrow \tilde{F} + A_i (1 + \alpha h_{iP}) m_i n^T$   $\triangleright$  build-plate flow (virtual ground node)
16:    end if
17:  end for
18:   $\tilde{F}_s \leftarrow (\tilde{F} + \tilde{F}^T)/2$ 
19:   $\tilde{R} \leftarrow \max(0, -n^T \tilde{F}_s n)$   $\triangleright = R + B$ , Eq. (16)
20:   $J(n) \leftarrow \tilde{R} + P$ 
21:  (singular values  $\sigma_1, \sigma_2, \sigma_3$  are computed from the face-to-face subtensor  $F$  only as auxiliary features for rank tuning)
22: end for
23: Generate refined directions around the best coarse candidates under  $J(n)$ 
24: Evaluate  $J(n)$  and feature vector  $x(n)$  for all coarse and refined candidates
25: Rank-normalize each feature inside the candidate set
26: Compute final tuning score  $T(n)$ 
27: Return directions sorted by  $T(n)$  with angular non-maximum suppression

```

In the implementation, instead of accumulating the build-plate term into the tensor, it is summed separately as a scalar B to equivalently compute $\tilde{R} = R + B$ (the equivalence of §5.1, numerically

confirmed within 10^{-12}). The above pseudocode makes explicit the unified-tensor form that is mathematically the same as that scalar accumulation. The face-to-face subtensor F is the part of $\tilde{F}$ excluding the ground-node term, and is used only for computing the singular value features.

10 Comparison Targets and Evaluation Protocol

We first define the comparison targets and evaluation procedure used in the quantitative analyses up to §13. The comparison targets are as follows.

1. PCA-based direction
2. Existing Support Tensor eigenvector direction
3. Initial tensor-eigenvector direction
4. This study’s Support Flow Tensor Field direction
5. TOMO_INT3 or commercial-slicer-based v_{ss} optimal direction

For each mesh, we generate a v_{ss} contour at 1° intervals under a critical angle of 60° , and evaluate how close the v_{ss} value at our method’s top candidate directions is to the global optimum. Even if the direction coordinates do not match exactly, if the v_{ss} values are similar we regard them as optimal orientations of the same level. Also, considering the case where the global optimum appears scattered across multiple basins, we compare angularly separated top- k basin representatives rather than simple top- k grid points. The candidate basins proposed by SFTF are re-input to TOMO_INT3 as a single initial orientation with angle step 0° to directly measure the Mss, and based on that measured value we mark the measured optimal and measured worst basins.

11 Comparison with Stored TOMO_INT3 Results

To verify the current implementation, we used the `*_tomo_int3_vss_grid_1deg_60deg.npz` files provided with the validation data. Each file contains the TOMO_INT3 support volume v_{ss} computed on a yaw–pitch 1° grid, and the global minimum direction of that grid. In the comparison process, instead of recomputing the TOMO grid, we mapped the top directions proposed by the current Support Flow Tensor Field onto the TOMO grid, and read the v_{ss} value at that location to compare with the global optimum.

The TOMO yaw–pitch corresponding to an SFTF direction n was chosen as the grid point that best satisfies, for TOMO’s rotation matrix $R(yaw, pitch, 0)$,

$$R(yaw, pitch, 0)n \approx e_z \quad (26)$$

Since a direction vector and its sign reversal can represent the same build axis, both n and $-n$ were evaluated and the smaller v_{ss} value was used as the signed result.

Table 1: Comparison of the stored TOMO_INT3 v_{ss} grid with the current SFTF candidates. Ratio is the ratio of the SFTF candidate’s v_{ss} to the TOMO global optimal v_{ss} .

Mesh	TOMO best	SFTF score1	Ratio	SFTF best-of-3	Ratio
Bunny 69k	3201.39	4665.85	1.46	4665.85	1.46
manikin	4657.72	8675.96	1.86	8675.96	1.86
dragon 100k	24446.98	45804.41	1.87	39547.57	1.62
happy 50k	2112.47	25467.94	12.06	23727.49	11.23
lucy 50k	5036.14	14192.68	2.82	14192.68	2.82

As Table 1 shows, SFTF with rank tuning applied selected candidates closer to the TOMO optimum than the existing raw score for Bunny, manikin, and dragon. In particular, Bunny’s signed score1 ratio improved from about 3.03 to 1.46, manikin from about 3.99 to 1.86, and dragon’s best-of-3 signed ratio from about 2.01 to 1.62. By contrast, the improvement was limited for happy and Lucy.

However, the rank tuning results in Table 1 are in-sample values measured on the very mesh set on which the weights were calibrated. Therefore these numbers are not generalization performance but a fit to the calibration set, and whether the weights are overfit and the performance on unseen meshes are evaluated separately by the cross-validation of §12.3. Also, how much each feature predicts the actual v_{ss} , and whether the asymmetric tensor term actually contributes to the result, are analyzed in §12.1.

11.1 Marking the Optimal Orientation Based on Top- k Basins Rather Than Top- k Points

In the yaw-pitch contour of TOMO.INT3, the global optimum may appear not as a single isolated point, but at almost the same value in several mutually separated basins. For example, in one mesh both the vicinity of $(yaw, pitch) = (69^\circ, 130^\circ)$ and the vicinity of $(265^\circ, 49^\circ)$ corresponded to the global optimum or a near-global optimum. In this case, simply marking the top 3 grid points with small v_{ss} values would repeatedly select only neighboring grid points within the same basin, failing to convey to the user the information that “other good orientation groups also exist.”

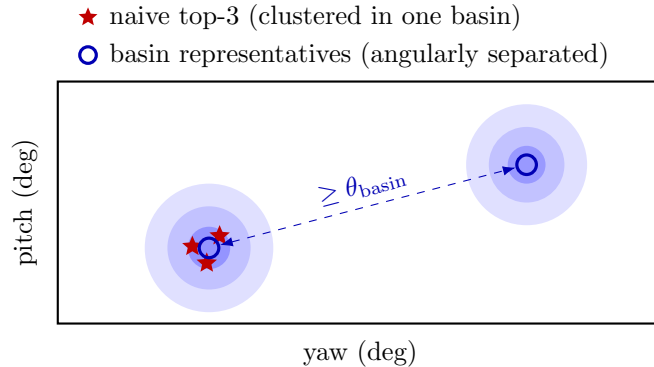


Figure 4: Top- k points vs. Top- k basins. When there are several mutually separated low-cost basins (blue regions) on the yaw-pitch cost landscape, simply picking the top points with small v_{ss} (red stars) repeatedly selects only neighboring points within one basin. With angular non-maximum suppression (NMS), keeping only candidates that are at least θ_{basin} away from the already-chosen representatives (blue circles), different near-optimal orientation groups are presented as representatives.

Therefore, the current visualization and post-processing mark the top basins (top- k basins) rather than the top points (top- k points). After converting a yaw-pitch point (ψ, ϕ) to a build direction $n(\psi, \phi)$, the distance between two orientations is defined as

$$d(n_a, n_b) = \cos^{-1}(n_a \cdot n_b) \quad (27)$$

Here we use $n_a \cdot n_b$ rather than $|n_a \cdot n_b|$. This is because, in the yaw-pitch orientation of TOMO.INT3, n and $-n$ can represent different build orientations. Scanning the sorted candidate list in order, a candidate whose angular difference from an already-selected basin representative satisfies

$$d(n, n_r) < \theta_{\text{basin}} \quad (28)$$

is considered the same basin and removed. The current implementation used $\theta_{\text{basin}} = 12^\circ$.

For the TOMO_INT3 contour itself, after sorting the v_{ss} grid in ascending or descending order, we apply the angular non-maximum suppression¹² above to select 3 optimal-basin representatives and 3 worst-basin representatives respectively. This approach shows, when the global optimum exists in multiple places, different optimal orientation groups instead of redundant grid points of the same basin.

The same basin perspective is applied to the SFTF candidates as well. First, we sort candidates with low and high final tuning score $T(n)$ respectively, but we remove same-basin duplicates to select low-score basin candidates and high-score basin candidates respectively. The current implementation keeps up to 40 basin candidates per direction. Then we put each candidate as the initial yaw-pitch orientation of TOMO_INT3, set the angle step to 0, and compute only a single orientation. That is, for candidate $q_k = (\text{yaw}_k, \text{pitch}_k)$ we directly measure

$$\text{Mss}_k = \text{TOMO_INT3}(q_k; \Delta\theta = 0) \quad (29)$$

Finally, we re-sort the candidate basins by this Mss_k value, and mark the 3 with small measured Mss as measured optimal basins and the 3 with large measured Mss as measured worst basins.

This procedure does not interpret the order of the SFTF scores directly as the final optimal/worst. Instead, SFTF plays the role of quickly proposing a broad candidate basin, and TOMO_INT3 is used as an accurate single-orientation verifier for a small number of candidates. Therefore the user can confirm not only a single optimal orientation, but several orientation groups that are mutually separated yet have a small actual support mass. This is especially useful when, in build orientation optimization, “which selectable optimal orientation groups exist” is more important than the coordinates of the rank-1 itself.

In an additional analysis, we mapped all coarse/refined candidates onto the TOMO grid to check the lowest v_{ss} within the candidate pool. With the default 512 coarse directions, the candidate pool of happy 50k itself did not sufficiently include the vicinity of the TOMO optimum. Increasing the number of coarse directions to 2048 lowered the lowest ratio within the candidate pool of happy 50k to about 1.91. This means that the failure of happy 50k is not merely a score tuning problem, but also includes limitations of the direction-sampling resolution and the candidate-generation method. However, in the 2048-candidate pool there was difficulty in stably selecting a good candidate with the current features alone. Good candidates were generally located in an intermediate region where the build-plate penalty is low while the pair flow and hit count are not too low. Therefore, in the future, rather than a simple linear rank score, a feature that directly reflects the face-to-face self-support coverage or a TOMO-style support-volume proxy is needed.

The main improvement factors confirmed in this experiment are as follows.

- The sign of the overhang normal and the build direction sign were directly reflected.
- Face-to-face internal visibility was evaluated via ray casting.
- A valid receiver face condition was imposed to consider only actually supportable faces.
- Overhangs that must be directly supported all the way to the build plate were handled as a separate penalty.
- Instead of 3 eigenaxes, spherical sampling and local re-search were used.
- In the final candidate selection, rank-normalized feature tuning based on comparison with the TOMO_INT3 grid was applied.

¹²Non-maximum suppression (NMS): a technique that adopts candidates from the best-scoring one in turn, but removes candidates too close to an already-adopted one to eliminate duplicates (originating from object detection). Here “closeness” is defined as the angle between two orientations, to prevent neighboring directions of the same basin from being redundantly selected.

- In the final visualization, angularly separated top- k basin representatives were marked rather than top- k points, providing the user with different near-optimal orientation groups.

At the same time, these results suggest that the current SFTF is useful not so much as a complete replacement of the TOMO_INT3 optimum, but as a candidate generator or warm-start method for reducing the expensive yaw-pitch global search. In particular, after proposing a few top candidates, locally verifying only their vicinity with TOMO or a slicer can greatly reduce the overall search cost.

12 Method Analysis: Feature Contribution, Cross-Validation, Comparison with Existing Methods

In this section we quantitatively analyze the validity of the proposed method from three perspectives. All analyses reuse the stored 1° TOMO_INT3 v_{ss} grid and do not require GPU re-computation, and are reproducible with the public analysis scripts. Unless otherwise noted, the best-of-3 ratio is the ratio, relative to the TOMO global optimum, of the smallest signed v_{ss} among the top 3 candidates (3° angular NMS) when mapped onto the TOMO grid, and smaller is better.

12.1 Feature Correlation and Objective Function Ablation

First, how much each feature predicts the actual v_{ss} was evaluated by the feature- v_{ss} Spearman rank correlation¹³ over the entire candidate pool (Table 2).

Table 2: Spearman rank correlation between features and TOMO v_{ss} over the entire candidate pool. The larger the positive value, the more valid the feature is as a cost.

Feature	Bunny	manikin	dragon	happy	lucy	Average
$J = R+P+B$	0.46	0.26	0.80	0.27	0.57	0.47
R (Rayleigh)	0.40	-0.33	-0.57	-0.03	0.14	-0.08
P (pair flow)	-0.04	-0.38	-0.72	0.00	-0.59	-0.34
B (build-plate)	0.50	0.66	0.85	0.29	0.62	0.58
H (hit count)	0.24	-0.53	-0.73	-0.00	-0.71	-0.35
S (nuclear)	-0.02	-0.36	-0.70	-0.05	-0.26	-0.28
σ_1	-0.03	-0.34	-0.66	-0.07	0.07	-0.21

Only the build-plate term $B(n)$ has a consistent positive correlation across all meshes (0.29–0.85, average 0.58), even higher than the predictive power of the entire objective function J (average 0.47). By contrast, P , S , σ_1 derived from the asymmetric tensor and the hit count H have negative average correlations and flip sign from mesh to mesh. That is, these terms do not generalize as cost terms with a fixed sign.

To re-confirm this at the objective-function level, we sorted candidates by several objective-function variants and then measured the best-of-3 ratio (Table 3).

¹³Spearman rank correlation coefficient: a correlation computed after converting two variables into *ranks* instead of values, where +1 means a perfectly monotonically increasing relationship and -1 a perfectly monotonically decreasing one. It measures whether “large values go together and small values go together” even if the relationship is not linear. See: [27]

Table 3: Best-of-3 ratio of the objective-function ablation (lower is better). The proposed objective function $J = R + P + B$ and the old objective function including nuclear/ σ_1 are completely identical per mesh.

Ranking variant	Bunny	manikin	dragon	happy	lucy	Average
T (rank tuned, in-sample)	1.46	1.86	1.62	11.23	2.82	3.80
$J = R + P + B$ (proposed)	2.47	3.99	2.01	11.15	2.82	4.49
$R+P+B+0.05S+0.05\sigma_1$ (old objective)	2.47	3.99	2.01	11.15	2.82	4.49
B alone	2.57	2.60	1.94	16.04	2.82	5.19
$0.05S + 0.05\sigma_1$ (tensor alone)	5.82	5.38	6.57	11.86	5.62	7.05
P alone	5.82	4.92	6.27	12.07	7.51	7.32

Crucially, the results of the proposed objective function $J = R + P + B$ and the old objective function including the nuclear/ σ_1 terms are identical across all meshes. This means that the nuclear/ σ_1 terms with weight 0.05 essentially do not contribute to the final candidate selection, and is the basis for excluding these terms from the objective function in this revision. Also, the variants using only tensor-derived terms ($0.05S + 0.05\sigma_1$, P alone) have the worst ratios. That is, the predictive power of our method originates not from the asymmetric tensor structure but from the build-plate penalty and the internal visibility evaluation, and the tensor terms play only an auxiliary role.

12.2 Reformulation into a Unified Tensor: Rayleigh Equivalence of the Build-Plate Term

The ablation in §12.1 showed that the build-plate term B takes almost full charge of the predictive power and that the Rayleigh term R derived from the tensor flips sign from mesh to mesh (Table 2). However, according to the virtual ground node interpretation of §5.1, B itself is a Rayleigh contribution of the augmented flow tensor, and the unified Rayleigh cost is exactly expressed as $\tilde{R} = R + B$ (Eq. (16)). This section verifies whether that unified term is actually a better predictor.

First, we confirmed the equivalence numerically. Computing the Rayleigh cost of the augmented tensor $\tilde{F}(n)$ directly over the entire candidate pool of the five stored meshes, it was identical within floating-point error, with $|\tilde{R} - (R + B)| < 10^{-12}$ for all candidates.

Table 4 compares the Spearman correlation of the unified term $\tilde{R}$ with the single terms. With R alone the average is -0.08 (unstable sign), but $\tilde{R} = R + B$ with the ground node added has an average of 0.60 , the best among single features, even higher than the current objective function $J = R + P + B$ (average 0.47). The reason J is lower than $\tilde{R}$ is that the face-to-face support volume P (average correlation -0.34 , Table 2) acts as a negative signal and lowers the predictive power when summed.

Table 4: Spearman correlation of the unified Rayleigh term $\tilde{R} = R + B$ and single terms with TOMO v_{ss} (higher is better). $\tilde{R}$ was computed directly from the augmented tensor and agrees with $R + B$ within 10^{-12} .

Mesh	R	B	$\tilde{R} = R+B$	$J = R+P+B$
Bunny	0.40	0.50	0.52	0.46
manikin	-0.33	0.66	0.65	0.26
dragon	-0.57	0.85	0.89	0.80
happy	-0.03	0.29	0.31	0.27
lucy	0.14	0.62	0.62	0.57
Average	-0.08	0.58	0.60	0.47

This was also confirmed by the best-of-3 ratio (Table 5). Sorting candidates by the unified term alone (i.e., removing P), the average ratio over the 4 meshes with normal candidate generation improves from J 's 2.82 to $\tilde{R}$'s 2.50. happy is a separate case where the candidate pool itself does not include a near-optimal (§12.5), and on this mesh the unified term is rather worse, but this is due to a sampling limitation, not scoring.

Table 5: Best-of-3 ratio when sorting candidates by $J = R + P + B$ and the unified term $\tilde{R} = R + B$ (lower is better). On the average excluding happy (candidate generation failure, §12.5), $\tilde{R}$ outperforms J .

Mesh	$J = R+P+B$	$\tilde{R} = R+B$
Bunny	2.47	2.57
manikin	3.99	2.89
dragon	2.01	1.70
lucy	2.82	2.82
Average (excl. happy)	2.82	2.50

In summary, the unified tensor reformulation (i) formulates the build-plate cost as a term of the flow tensor, justifying the “tensor” representation, and (ii) shows that the single-tensor Rayleigh cost $\tilde{R}$ with the negative signal P removed is consistently superior to the existing J .

Verification of the Height Convention. The ground term uses a height convention opposite in sign to the face-to-face term (amplification instead of attenuation). To show that this asymmetry is not arbitrary, we compared four plate conventions. The current amplification $B_{\text{amp}} = \sum O_i A_i (1 + \alpha h)$, the height-free $B_{\text{flat}} = \sum O_i A_i$, the attenuation $B_{\text{att}} = \sum O_i A_i / (1 + \alpha h)$ symmetric with the face-to-face term, and $B_{\text{uni}} = \sum O_i^2 A_i / (1 + \alpha h)$ that fully follows the face-to-face weight law. The average Spearman correlations were 0.58, 0.51, 0.43, 0.47 respectively, so *the current amplification convention is the highest and most consistent*, and the attenuation convention B_{att} matched symmetrically with the face-to-face term was the lowest (negative correlation on happy). That is, the sign asymmetry “distant self-support is weakened, tall build-plate support is strengthened” is physically correct and empirically supported. However, the fully unified convention B_{uni} greatly improves best-of-3 on slender shapes such as manikin (e.g., manikin $2.89 \rightarrow 1.37$), leaving room for a shape-adaptive convention in the future.

12.3 Cross-Validation of Rank Tuning

Since the rank tuning weights in Table 1 were calibrated together on the reported meshes, to evaluate generalization performance we performed Leave-One-Mesh-Out (LOMO) cross-validation.¹⁴ Setting each mesh as hold-out and refitting the weights on the remaining meshes (ridge regression¹⁵ on the intra-mesh rank-normalized features, with the target being the intra-mesh rank-normalized v_{ss}), we measured the best-of-3 ratio of the hold-out mesh. Excluding happy, for which candidate generation itself fails (see the end of §12.3 below and the oracle), we evaluated on the 4 meshes whose candidate pool includes a near-optimal direction.

Table 6: LOMO cross-validation best-of-3 ratio of rank tuning (lower is better). The in-sample column is the calibration-set fit value reported in Table 1. The oracle is the best direction existing within the candidate pool, representing the upper bound of candidate generation.

Mesh	oracle	untuned J	in-sample (reported)	LOMO
Bunny	1.46	2.47	1.46	2.20
manikin	1.13	3.99	1.86	2.62
dragon	1.29	2.01	1.62	1.86
lucy	1.02	2.82	2.82	2.82
Average	1.22	2.82	1.94	2.37

The reported in-sample average 1.94 is not reproduced in honest cross-validation, and the hold-out average is 2.37, about 0.43 (about 22%) higher. This difference is the optimistic bias of the weight calibration. Also, among the 7 weights, only R (≈ 0.7) and B (≈ 1.0) are reproduced stably across folds, while the reported large $P = -2.39$, $H = 2.31$, $S = -2.29$, $\sigma_1 = 1.64$ are not reproduced in any fold. In fact, the LOMO average ratio of reduced models using only stable features is B alone 2.48, $R + B$ 2.46, $R + P + B$ 2.44, not much different from the full 7-feature model (2.37), and all are superior to untuned J (2.82). Therefore, instead of 7 calibrated weights, tuning a few features centered on R , B (and P if needed) is simpler yet more robust. Meanwhile, the fact that the hold-out ratio is clearly larger than the oracle (1.22) in all variants suggests that the room for improvement lies not in candidate generation but in the scoring stage.

12.4 Unified Comparison with Existing Methods

Finally, under the same best-of-3 ratio criterion we compared PCA directions, Support Tensor eigenvector directions, and SFTF (untuned and LOMO cross-validation tuned) (Table 7). PCA uses the principal axes of the area-weighted face-centroid covariance, and Support Tensor uses the eigenvectors of $M = \sum_i A_i m_i m_i^T$, taking the best value over the 3 axes of each method (including $\pm$). The SFTF-tuned column is not an in-sample value but the LOMO value of §12.3.

¹⁴Cross-validation: a method that sets aside part of the data from training and uses it only for validation, to estimate performance on unseen data. LOMO is a “leave one mesh out” scheme that excludes one mesh, fits the weights on the rest, and then measures performance on the excluded mesh. See: [28]

¹⁵Ridge regression: linear regression that adds an L_2 penalty so that the regression weights do not become too large, reducing overfitting. See: [12]

Table 7: Unified comparison of existing methods and SFTF (best-of-3 signed v_{ss} ratio, lower is better). SFTF-tuned is the cross-validated generalization estimate.

Mesh	PCA	Support Tensor	SFTF untuned	SFTF tuned (LOMO)	oracle
Bunny	2.82	3.71	2.47	2.20	1.46
manikin	2.65	2.50	3.99	2.62	1.13
dragon	2.69	2.33	2.01	1.86	1.29
lucy	6.37	8.67	2.82	2.82	1.02
Average	3.63	4.30	2.82	2.37	1.22

Cross-validated SFTF (average 2.37) is about 35% better than PCA (3.63) and about 45% better than Support Tensor eigenvectors (4.30), and even untuned J (2.82) is better than the two existing methods. In particular, the Support Tensor eigenvectors are worse than even PCA (8.67 on lucy), which quantitatively supports this study’s starting assumption that the statistical principal axes of the normal distribution alone cannot explain the direction-dependent support volume. This comparison shows that even when discarding the reported in-sample numbers and using the conservative cross-validation numbers, SFTF’s advantage over existing methods is maintained.

12.5 Additional Robustness Analysis: Critical Angle, Sampling Density, Dimensional Normalization

Critical Angle Matching. SFTF’s overhang definition $O_i(n) = \max(0, -m_i \cdot n)$ includes all downward faces regardless of the critical angle, but TOMO-INT3 regards as support-needing only the faces with $-m_i \cdot n > \cos \theta_c$ among downward faces ($m_i \cdot n < 0$). As a result of applying the same gate to the SFTF overhang to match the definitions, under the $\theta_c = 60^\circ$ setting (same as TOMO) the LOMO average best-of-3 ratio worsened from 2.37 to 2.82, and at $\theta_c = 45^\circ$ it was 2.42, similar to the baseline. That is, matching the critical angle gate does not provide a consistent improvement, which is interpreted as the rank tuning already absorbing the difference in the overhang definition. Therefore the default implementation does not apply the gate.

Sampling Density and Candidate Generation Limit. Increasing the number of coarse Fibonacci sphere directions, we measured the best value within the candidate pool (oracle ratio) (Table 8).

Table 8: Oracle (best within candidate pool) ratio by the number of coarse directions. Meshes other than happy already include a near-optimal direction at 512.

# coarse directions	Bunny	manikin	dragon	happy	lucy
512	1.46	1.13	1.29	4.35	1.02
1024	1.50	1.11	1.33	3.94	1.02
2048	1.61	1.06	1.28	1.91	1.04
4096	1.62	1.13	1.27	1.91	1.30

happy’s oracle ratio is 4.35 at 512, with no near-optimal direction included in the candidate pool. Increasing the number of directions to 2048 lowers it to 1.91, partially resolving the candidate-generation limit. By contrast, other meshes already have an oracle of 1.0–1.5 at 512, with no benefit from increasing the number of directions. However, even when its candidates are reinforced, happy’s best-of-3 ratio of untuned J remains at 10.51 even at 4096, showing limitations in both sampling and scoring. This is the basis for separating happy as a distinct hard case.

Dimensional Normalization of the Objective Function. The terms of the objective function $J = R + P + B$ have different units ($R, P \sim \text{area}^2$, $B \sim \text{area}$). When the three terms are normalized to the same scale within the candidate pool and summed with equal weights, the Spearman correlation of raw J drops greatly from 0.52 to 0.14 under min-max normalization¹⁶ and to 0.10 under z-score normalization,¹⁷ and the best-of-3 ratio also worsens from 2.82 to 3.53 and 3.86 respectively. This shows that the predictive power of raw J originates not from a balance of several terms but from the dominance of the build-plate term, which has the largest scale. Therefore the physical meaning of the weights is more properly secured not by simple dimensional normalization, but by removing non-predictive terms and reducing to a focus on R , B (§12.3).

13 Verification against Four Implementations on a Controlled Five-Group Shape Set

While the preceding sections addressed the accuracy and generalization on stored Stanford mesh grids, this section directly contrasts SFTF’s *algorithmic characteristics* on a difficulty-controlled shape set with the four implementations TOMO_INT3, TOMO_CUDA, Python SFTF, and SFTF_C++. The verification was performed as a batch experiment; SFTF generates a candidate pool from the input mesh alone (coarse 512 directions, candidate pool 1292), while TOMO_INT3 and TOMO_CUDA performed an exhaustive sweep on the same mesh over a yaw-pitch 3° grid ($120 \times 120 = 14,400$ directions), critical angle 60° , via DLL.¹⁸ SFTF_C++ is an implementation that ports the same candidate-generation algorithm as Python SFTF to a C++ DLL, and the measured value is the pure computation time of the candidate generator. All four implementation times are available for all 35 meshes A–E.

13.1 Experimental Setup

The dataset consists of five groups that deliberately separate and control the two axes of *shape difficulty* and *mesh scale*. A–C set the scale in the $\sim 10^1$ – 10^5 -face range and vary the difficulty axis from basic shapes to freeform organic shapes, revealing the *fixed-cost floor* of the exhaustive sweep (a cost almost independent of mesh complexity). D–E fix the difficulty as real manufacturing parts while pushing the scale axis from 256 faces to 1.04×10^6 faces, revealing the *adaptive cost* by which the SFTF candidate-evaluation cost increases adaptively to mesh complexity. Therefore A–E is not a single size ramp, and the partial overlap of the face-count ranges of groups C and D is the result of a design that separates and controls the two axes, not an ordering error.

- **A (basic shapes, 5; lower difficulty):** cube, sphere, cylinder, cone, torus (face count $\sim 10^1$ – 10^3).
- **B (simple functional parts, 5; mid difficulty):** u_bracket, hook, c_clamp, pipe_elbow, hollow_box.
- **C (organic/freeform benchmarks, 5; upper difficulty):** standard scan/sculpture meshes such as Bunny (69,662 faces), manikin, dragon ($\sim 10^5$ faces), happy, Lucy. The Stanford grid meshes of the preceding sections are exactly this group.

¹⁶Feature scaling (normalization) that linearly maps the minimum to 0 and the maximum to 1. See: [11]

¹⁷Standard score. A normalization that subtracts the mean and divides by the standard deviation to make “mean 0, variance 1.” See: [11]

¹⁸To handle a large-scale batch that sweeps 35 meshes over 5 critical angles ($\theta_c \in \{0, 30, 45, 60, 90\}^\circ$) with two backends, the sweep grid of this verification was set to a coarser 3° than the prior GPU-MSST’s 1° ($360 \times 360 = 129,600$ directions). On a denser 1° grid, the TOMO sweep time increases about $9\times$, widening the speed gap relative to SFTF further.

- **D (Thingi10K set 1, 10; lower scale)**: real manufacturing parts with 256–88,106 faces [31].
- **E (Thingi10K set 2, 10; upper scale)**: large high-resolution meshes with about 5.1×10^5 – 1.04×10^6 faces.

For the 35 meshes A–E, we recorded the Python SFTF candidate-evaluation time, the TOMO_INT3 DLL sweep time, the TOMO_CUDA DLL sweep time, the SFTF_C++ candidate-evaluation time, the hit count and candidate score of the SFTF top direction, and the TOMO contour HTML. This section addresses the contrast of computational structure and implementation axes rather than the direction-accuracy ratio, and the quantitative analysis of direction accuracy is handled by the grid comparison of §12.1–§12.4.

13.2 Overall Computation Time Comparison

Table 9 summarizes the four-implementation measurements of the 35 meshes. The most prominent characteristic is the contrast of *fixed cost vs. adaptive cost*. TOMO_INT3 takes about 4 seconds even for a cube with only 12 faces, having a large fixed cost almost independent of mesh complexity due to the nature of the yaw–pitch exhaustive sweep (because even on a 3° grid it evaluates all 14,400 directions). By contrast, the cost of SFTF adapts to mesh complexity, remaining 0.12–0.74 seconds for basic shapes, 1.0–3.2 seconds for organic benchmarks (C), 0.36–3.42 seconds for Thingi10K parts (D), and only 9.71–22.11 seconds even for large meshes of 500k–1M faces (E). Over all 35, Python SFTF is 1.4–34 $\times$ faster than TOMO_INT3 (3°), with an average speed ratio of about 9 $\times$. SFTF_C++ is additionally 3.9–233 $\times$ faster than Python SFTF, and 10–5,160 $\times$ faster than TOMO_INT3 (3°). That is, SFTF is suitable not as a replacement for the expensive global sweep, but as a candidate generator that proposes promising orientations from several to several thousand times faster in front of it, and the denser the sweep grid is made to 1° , the wider this gap becomes.

The result of TOMO_CUDA is more subtle. On the simple shapes of A/B, TOMO_CUDA is generally faster than TOMO_INT3 and shows the GPU-acceleration effect of the exhaustive sweep. However, on the organic and large meshes of C/D/E, TOMO_CUDA is often slower than TOMO_INT3, and on the overall average the TOMO_CUDA/TOMO_INT3 time ratio is about 1.07. This means that in this verification environment, the input preparation of the exhaustive sweep, the mesh complexity, and the DLL-internal data structures and memory-movement cost can govern performance more than the GPU backend itself. Therefore the core speed gain of this paper comes not from the “hardware axis of changing INT3 to CUDA” but from the “algorithmic axis of reducing 14,400 exhaustive directions on a 3° grid (129,600 at 1°) to a 1292-candidate evaluation.”

Table 9: Computation time of the four implementations for the 35 meshes of the five groups. t_{INT3} and t_{CUDA} are the TOMO INT3/TOMO.CUDA 3° exhaustive sweep ($\theta_c = 60^\circ$) DLL times respectively, t_{py} is the Python SFTF candidate evaluation, and $t_{\text{C++}}$ is the SFTF_C++ pure computation time. hit is the number of valid face-to-face supports of the Python SFTF top direction. The speed columns are $t_{\text{INT3}}/t_{\text{py}}$ and $t_{\text{INT3}}/t_{\text{C++}}$ respectively.

ID	Shape	Faces	hit	$t_{\text{INT3}}(\text{s})$	$t_{\text{CUDA}}(\text{s})$	$t_{\text{py}}(\text{s})$	$t_{\text{C++}}(\text{ms})$	INT3/py	INT3/C++
<i>A: basic shapes</i>									
A1	cube	12	0	4.12	1.26	0.13	4.1	32×	1,006×
A2	sphere	2,976	498	4.29	1.53	0.74	23.9	6×	179×
A3	cylinder	192	0	4.12	2.16	0.17	1.3	24×	3,172×
A4	cone	96	0	4.13	1.47	0.12	0.8	34×	5,157×
A5	torus	2,304	184	4.28	1.48	0.45	14.2	10×	301×
<i>B: simple parts</i>									
B1	u_bracket	28	2	4.13	1.35	0.15	3.9	28×	1,059×
B2	hook	1,560	235	4.21	1.44	0.58	15.8	7×	266×
B3	c_clamp	28	1	4.13	1.63	0.17	0.8	24×	5,157×
B4	pipe_elbow	2,400	500	4.27	1.49	0.81	25.3	5×	169×
B5	hollow_box	24	4	4.13	1.64	0.21	0.9	20×	4,592×
<i>C: organic benchmarks</i>									
C1	Bunny	69,662	81	6.17	6.97	1.69	247.7	4×	25×
C2	manikin	13,672	134	4.38	3.70	1.02	44.3	4×	99×
C3	dragon	99,999	970	9.65	12.11	3.17	323.3	3×	30×
C4	happy	49,944	240	5.92	6.63	1.50	130.1	4×	45×
C5	lucy	49,999	542	6.44	8.87	1.87	172.5	3×	37×
<i>D: Thingi10K set 1</i>									
D1	37009	23,909	57	5.76	13.28	1.37	114.5	4×	50×
D2	37010	13,247	35	4.95	13.28	1.24	63.5	4×	78×
D3	37011	17,288	82	5.06	5.81	1.15	89.2	4×	57×
D4	37091	256	42	4.13	2.00	0.36	5.2	11×	795×
D5	37095	468	65	4.17	3.62	0.49	6.2	9×	672×
D6	37111	88,106	386	12.33	21.37	3.42	465.4	4×	26×
D7	37137	64,078	434	10.18	22.16	2.90	305.5	4×	33×
D8	37402	8,322	119	5.00	6.82	2.43	49.7	2×	101×
D9	37415	13,340	77	5.73	11.52	2.96	65.4	2×	88×
D10	37416	12,036	496	4.74	6.10	3.39	80.4	1.4×	59×
<i>E: Thingi10K set 2, large meshes</i>									
E1	39507	969,520	562	129.39	131.73	16.92	4387.6	8×	29×
E2	45809	517,589	107	64.89	73.88	9.71	1984.5	7×	33×
E3	45811	1,037,876	1,309	132.74	145.57	18.69	4515.6	7×	29×
E4	46012	638,754	50	82.74	101.35	11.15	2583.8	7×	32×
E5	55280	900,490	1,072	118.42	138.23	16.78	3957.9	7×	30×
E6	59197	786,432	699	99.04	105.32	17.50	4119.5	6×	24×
E7	61384	516,608	1,766	65.83	80.09	12.85	2065.0	5×	32×
E8	64445	514,938	2,288	21.98	31.69	15.05	2208.0	1.5×	10×
E9	64446	514,938	2,288	21.89	31.78	14.93	2211.4	1.5×	10×
E10	65942	754,688	1,175	108.07	111.99	22.11	3201.6	5×	34×

13.3 Group-wise Averages and Separation of Implementation Axes

Table 10: Group-wise average execution time and speed ratio. The SFTF_C++ average is in ms, and the rest are in seconds. A TOMO_CUDA/TOMO_INT3 ratio greater than 1 indicates a case where the CUDA backend is slower than INT3.

Group	py (s)	INT3 (s)	CUDA (s)	C++ (ms)	INT3/py	CUDA/INT3
A	0.32	4.19	1.58	8.9	6–34×	0.31–0.52
B	0.38	4.17	1.51	9.3	5–28×	0.33–0.40
C	1.85	6.51	7.66	183.6	3–4×	0.84–1.38
D	2.17	6.21	10.60	124.5	1.4–11×	0.48–2.68
E	15.57	84.50	95.16	3123.5	1.5–8×	1.02–1.45

13.4 Scalability to Large Meshes (Group E)

Group E consists of 10 large Thingi10K meshes with face counts from about 5.1×10^5 to 1.04×10^6 , examining whether SFTF works on high-resolution meshes beyond real-part scale. The key observation is the *gentle increase of the SFTF cost*. Even on E meshes whose face count is about 6–12× that of D6 (88,106 faces), Python SFTF candidate evaluation is 9.71–22.11 seconds, increasing only roughly sub-linearly with the face count. This is because the ray budget is fixed at an upper bound of 3000 via $K_{ray} = \text{clip}(0.02N_f, 500, 3000)$, so on large meshes the face preprocessing and BVH construction govern the cost while the ray casting cost saturates. SFTF_C++ finishes candidate evaluation on the same large meshes in 1.98–4.52 seconds, 3.9–6.9× faster than Python. These two candidate-generator costs are independent of the sweep grid (SFTF does not perform an exhaustive sweep).

By contrast, the 3° exhaustive sweep of TOMO_INT3 takes 22–133 seconds on group E (about 9×, 126–1,077 seconds, at 1°). There are relatively short cases of about 22 seconds on identical 514,938-face meshes such as E8/E9, but on most large meshes tens of seconds to over a hundred seconds are needed. TOMO_CUDA was also 32–146 seconds, and was not faster than INT3 on large meshes in this verification environment. This shows that the algorithmic reduction of the number of candidates provides a more stable speed gain even on large meshes than the hardware acceleration of the exhaustive-sweep backend.

Internal consistency is also confirmed. E8 (64445) and E9 (64446) are essentially identical meshes (514,938 faces), and SFTF produced the same top score (−0.9824), the same direction, and the same hits (2,288), showing deterministic reproducibility, with the only difference being the execution time (15.05 vs. 14.93 seconds). Also, the number of valid face-to-face supports (hit) increases as the mesh detail becomes richer (e.g., E7 1,766, E8/E9 2,288), suggesting that the internal self-support structure of high-resolution shapes is faithfully reflected in SFTF’s flow tensor.

13.5 Per-Direction Cost Structure Decomposition

Unlike TOMO_INT3, which summarizes each orientation with a single scalar v_{ss} , SFTF provides the cost of the top direction decomposed into the Rayleigh term R , the face-to-face support volume P , the build-plate penalty B , and the number of valid face-to-face supports hit (Table 11). This decomposition is a distinguishing characteristic of SFTF in that it directly provides an interpretation of “why that direction is good.”

Convex basic shapes such as cube, cylinder, and cone have $\text{hit} = 0$, $R = P = 0$, and the cost is composed entirely of the build-plate term B . This means the algorithm directly identifies that “there is no self-support relationship, and seating a plane on the floor makes support unnecessary.” Conversely, sphere, torus, and all organic/real parts have $\text{hit} > 0$, so a face-to-face self-support structure exists, and R, P, B contribute together. For example, D10 (37416) has

hit= 496, $P \approx 1.2 \times 10^4$, with face-to-face self-support dominant and $B \approx 1.3$, so the build-plate dependence is small, whereas D6 (37111) has $B \approx 1.6 \times 10^3$, so the build-plate support requirement is large. The same contrast appears in organic meshes: dragon has hit= 970 with R, P, B evenly distributed, whereas Bunny is dominated by $B \approx 176$. Such regime distinctions are not revealed by a single v_{ss} value alone.

Table 11: Cost decomposition of the SFTF top direction of representative meshes. Convex basic shapes (cube, cone) are pure build-plate support with hit= 0, $R = P = 0$, and the rest show face-to-face self-support (P) and build-plate support (B) together.

ID	Shape	hit	R	P	B
A1	cube	0	0	0	401.5
A4	cone	0	0	0	179.8
A2	sphere	498	47.3	109.5	0.02
A5	torus	184	15.9	40.3	209.1
B2	hook	235	19.9	55.4	128.7
C1	Bunny	81	1.4	25.2	176.3
C3	dragon	970	109.4	267.6	417.5
D6	37111	386	227.9	932.8	1,613
D10	37416	496	6,381	12,190	1.3

13.6 Support Volume and Optimal Orientation Change with the Critical Angle

Even for the same shape, which faces are classified as overhangs changes with the printable critical angle θ_c , so the TOMO support volume and its global optimal orientation also depend on θ_c . To quantify this, we re-swept all 35 meshes over the five critical angles $\theta_c \in \{0, 30, 45, 60, 90\}^\circ$ with TOMO.INT3 and TOMO.CUDA (a 3° grid). For each critical angle, we generated a four-implementation summary table and a Polyscope overlay that captures the best/worst orientations of the five groups in one frame (Figure 5).

Table 12 shows that the TOMO.INT3 global minimum v_{ss} of representative meshes *decreases monotonically* as θ_c increases. This is consistent with intuition: as θ_c grows, even steeper faces are allowed as self-support, so the support requirement decreases. The pattern of the decrease differs by shape. cube shows a *threshold behavior*: for $\theta_c \leq 30^\circ$ the optimal orientation stands an edge up and $v_{ss} > 0$, but for $\theta_c \geq 45^\circ$ a plane is seated on the floor and $v_{ss} = 0$; hook decreases smoothly from $1,736 \rightarrow 0$. For large/organic meshes, the global optimal *orientation* itself shifts; for example, the optimal (*yaw, pitch*) of 37009 jumps from near $(\approx 345^\circ, 0^\circ)$ for $\theta_c \leq 45^\circ$ to a different basin near $(\approx 150^\circ, 186^\circ)$ for $\theta_c \geq 60^\circ$.

Table 12: TOMO-INT3 global minimum v_{ss} by critical angle (representative meshes, 3° grid). The support volume decreases monotonically as θ_c increases. In the region of large θ_c , almost all faces become self-supporting and support is essentially unnecessary, in which case the numerical residual of the MSST decomposition $V_{ss} = V_{tc} - V_o - V_{nv}$ can make v_{ss} appear as a small negative number (effectively 0).

ID	Mesh	$\theta_c=0^\circ$	30°	45°	60°	90°
A1	cube	1,769	1,769	0	0	0
B2	hook	1,736	725	319	133	0
C1	Bunny	49,780	23,660	13,970	5,147	-7,857
C3	dragon	217,000	119,400	84,500	24,450	-18,090
D1	37009	65,810	4,729	1,061	-941	-7,226
D6	37111	99,200	58,980	38,740	9,703	-6,270
E1	39507	26,380	12,510	6,493	-6,337	-13,410
E7	61384	75,040	21,140	17,590	-3,503	-14,610

A contrast to note is that, while the TOMO optimal orientation shifts with θ_c above, the current SFTF candidates are *invariant* to θ_c . This is because the SFTF candidate generator is implemented with a setting that does not truncate the overhang coefficient $O_i(n) = \max(0, -m_i \cdot n)$ by the critical angle (`USE_CRITICAL_ANGLE=False`), and as a result the best/worst overlays of the five critical angles (Figure 5) are visually identical. Therefore Figure 5 shows the result of an arbitrary single critical angle ($\theta_c = 60^\circ$) as a representative, and θ_c -dependent candidate generation (SFTF with overhang truncation turned on) remains future work.

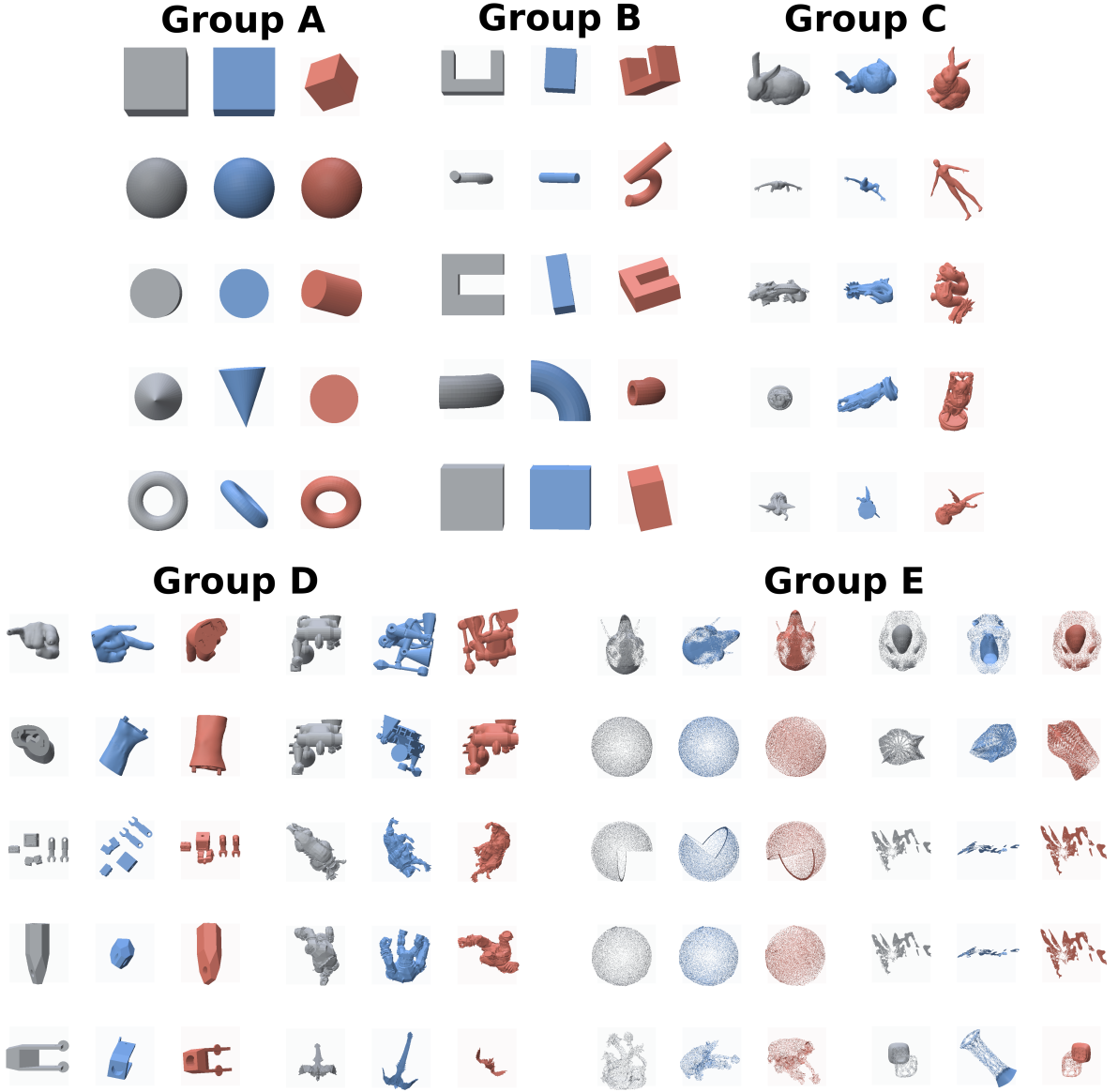


Figure 5: SFTF_C++ best/worst build-orientation Polyscope overlay arranged by group for the five groups (A–E) ($\theta_c = 60^\circ$). A label is placed atop each group, with groups A–C in the top row and groups D/E (a vertical 5-row 2-column layout each) in the bottom row, grouped about the page center. Each mesh shows the original (gray), the SFTF-score best orientation (blue), and the SFTF-score worst orientation (red) side by side, displayed in the actual print pose by rotating the stored build direction to world +Z. These are the lowest/highest SFTF-score candidates, *not* TOMO-verified optima. Since SFTF candidates are invariant to θ_c , the overlays of the five critical angles are identical.

13.7 Agreement of the Optimal Orientation

On trivial cases the two methods agree. For cube, cylinder, cone, u_bracket, c_clamp, hollow_box, and pipe_elbow, the TOMO_INT3 global optimal $v_{ss} = 0$ (no support needed), and the SFTF top direction also selects an axis-aligned face-normal direction with a score nearly 0. That is, both methods arrive at the same conclusion that “seating a plane on the build plate makes support unnecessary.” On non-trivial organic/real parts (hook, groups C–D), the TOMO global optimum is located at an off-axis orientation with large v_{ss} , and SFTF likewise reflects the complex self-support structure with high hits and balanced R, P, B . However, this five-group experiment

focuses on the contrast of computational characteristics and structural interpretation rather than the quantitative ratio of accuracy, and the quantitative analysis of direction accuracy is handled by the grid comparison of §12.1–§12.4.

In summary, this five-group verification quantitatively confirms the following SFTF characteristics. (i) Over all A–E, Python SFTF achieves a $1.4\text{--}34\times$ computation-time reduction relative to TOMO.INT3 (3° sweep), and SFTF.C++ $10\text{--}5,160\times$ (the gap grows on a denser 1° grid), (ii) a cost that adapts to mesh complexity (absence of fixed cost) and scalability up to $\sim 10^6$ -face meshes, (iii) a per-direction $R/P/B$ /hit structure decomposition that a single v_{ss} cannot provide, (iv) agreement with the TOMO global optimum on trivial cases, (v) in the four-implementation comparison, that TOMO.CUDA is not always faster than INT3, and that the gain of the algorithmic axis of “reducing the evaluation targets” is more stable and larger than “changing the exhaustive-sweep backend,” (vi) that the C++ port enables candidate evaluation in ms units for small meshes and seconds for large meshes, (vii) that as the critical angle θ_c grows the TOMO support volume decreases monotonically and the global optimal orientation shifts, whereas the current SFTF candidates are invariant to θ_c (§13.6).

14 Conclusion

This study proposed a direction evaluation method that combines the Support Flow Tensor Field and a build-plate penalty for build-orientation optimization in additive manufacturing. The eigenvector interpretation of the existing symmetric tensor is useful for shape directionality, but it does not sufficiently reflect direction-dependent overhangs, internal visibility, and the build-plate support requirement.

The current algorithm directly computes an asymmetric Support Flow Tensor for each candidate direction, and constructs a basic objective function $J = R + P + B$ that combines the Rayleigh support cost, the face-to-face support volume, and the build-plate penalty. In the actual final candidate selection, it applies a rank-normalized tuning score T of $J, R, P, B, H, S, \sigma_1$ to a candidate pool made by coarse spherical sampling and small-angle refined sampling, to choose promising orientations to pass to slicer/TOMO verification.

The quantitative analysis (§12.1–§12.4) shows both the strengths and limitations of the method. Cross-validated SFTF is consistently superior to PCA and Support Tensor eigenvector directions, supporting the starting assumption. At the same time, most of the predictive power originates from the build-plate penalty and the internal visibility evaluation, the marginal contribution of some tensor-derived terms such as nuclear/ σ_1 is small, and the calibrated rank tuning weights show an optimistic bias on unseen meshes. Meanwhile, in §5.1–§12.2 we showed that the build-plate penalty is exactly equivalent to the Rayleigh contribution obtained by adding a virtual ground node to the flow tensor. This unifies the term driving the predictive power into the tensor contraction $\tilde{R} = R + B$, justifying the “tensor” formulation, and confirms that $\tilde{R}$, excluding the negative-signal face-to-face support volume P , is consistently superior to the existing J .

The present implementation should therefore be interpreted as a fast candidate generator rather than a complete support-volume solver. Its first limitation is that the support surrogate remains volume-centered. TOMO.INT3 evaluates support volume by explicitly separating exterior, interior or occluded, and bottom/build-plate-related structures before forming v_{ss} , whereas the current SFTF aggregates surface-flow, hit-count, and build-plate height cues into a simplified volumetric proxy. Consequently, SFTF can cheaply rank promising orientations, but it does not yet reproduce the decomposed support-volume accounting of TOMO.INT3. A second limitation is that the objective does not directly model slicer-specific support generation, support-removal accessibility, contact-scar area, surface quality, print time, thermal deformation, or material/process-dependent overhang behavior. A third limitation is methodological: SFTF reduces the number of directions to be verified, so its final accuracy still depends on whether

the candidate pool contains a direction near the true low-support basin. This was visible in the hard “happy” case and in the θ_c study, where the TOMO optimum changed with the critical angle while the present SFTF candidates remained invariant to θ_c .

These limitations define the next development steps. First, we will extend SFTF toward an exterior/interior/bottom-aware formulation calibrated against the separated TOMO_INT3 components rather than only against the final scalar v_{ss} . Second, we will remove terms with marginal contribution or replace them with features more directly correlated with the actual support volume, such as face-to-face self-support coverage and a TOMO-style support-volume proxy. Third, we will determine rank-tuning weights by cross-validation rather than by a calibration set, and normalize the dimensions of the objective-function terms to improve the physical meaning and transferability of the weights. Fourth, we will extend the scoring model to multi-objective build-orientation selection by adding projection area, print height, contact stability, support removability, contact-scar, and thermal-deformation-related terms. Finally, as observed in §13.6, we will implement θ_c -dependent candidate generation with overhang truncation turned on and adaptive sampling for cases where the initial candidate generation does not include a near-optimal direction.

A related downstream application is support-aware mesh partitioning. Although this paper focuses on build-orientation candidate generation, the same SFTF pass produces per-face quantities before tensor reduction, including overhang strength, receiver type, support height, incoming receiver count, and build-plate dependence. Retaining these quantities makes it possible to use SFTF as an adaptive face-feature extractor for grouping a mesh into regions with coherent support behavior or different preferred build directions. In our preliminary mesh-partitioning implementation, SFTF features are fused with face-center coordinates or used directly through support-flow region growing; the resulting partitions follow support classes more consistently than coordinate-only clustering on the tested example. Because part-wise TOMO/slicer verification, cut feasibility, assembly constraints, and mechanical strength are outside the scope of the present orientation-optimization study, we treat SFTF-driven mesh partitioning as a downstream application and leave its full validation to future work.

Appendix

A Derivation of the Ground-Node Rayleigh Equivalence

This appendix gives the explicit construction of the augmented flow tensor $\tilde{F}(n)$ and the term-by-term derivation of the identity $\tilde{R}(n) = R(n) + B(n)$ (Eq. (16)) stated in §5.1.

In the face-to-face flow term $m_i m_j^T$, the receiver normal m_j faces upward with $m_j \cdot n > \eta > 0$, and the support-face normal of the build plate is exactly the build direction n ($m \cdot n = 1$, i.e., the limiting receiver that faces most upward). Therefore, treating a face $i \in \mathcal{B}(n)$ supported by the plate as a “flow pair with a virtual node whose normal is n ,” we define the augmented tensor as follows.

$$\tilde{F}(n) = \underbrace{\sum_{(i,j) \in \mathcal{R}(n)} w_{ij}(n) A_i A_j m_i m_j^T}_{\text{face-to-face flow}} + \underbrace{\sum_{i \in \mathcal{B}(n)} A_i (1 + \alpha h_{iP}(n)) m_i n^T}_{\text{build-plate flow (virtual node)}}. \quad (30)$$

Since $m_i \cdot n = -O_i(n)$, the Rayleigh contribution of a single plate term is

$$n^T \left[\frac{1}{2} (m_i n^T + n m_i^T) \right] n = (m_i \cdot n) (n \cdot n) = -O_i(n), \quad (31)$$

and multiplying by the coefficient $A_i(1 + \alpha h_{iP})$ and summing over $i \in \mathcal{B}(n)$, the amount that the plate term contributes to $-n^T \tilde{F}_s n$ is exactly

$$\sum_{i \in \mathcal{B}(n)} O_i(n) A_i (1 + \alpha h_{iP}(n)) = B(n). \quad (32)$$

The face-to-face part of $\tilde{F}(n)$ contributes $R(n) = \max(0, -n^T F_s(n) n)$ in the same way, so the Rayleigh support cost of the augmented tensor holds exactly as Eq. (16),

$$\tilde{R}(n) = \max(0, -n^T \tilde{F}_s(n) n) = R(n) + B(n).$$

This equivalence was numerically confirmed as $\max |\tilde{R} - (R + B)| < 10^{-12}$ over the entire candidate pool of the five stored meshes (about 1,292 candidates each; §12.2). The implementation accumulates the plate term separately as the scalar B rather than into the tensor, which by the above identity is mathematically equivalent to evaluating the Rayleigh cost of $\tilde{F}(n)$ directly.

B Implementation Details of Rank-Based Candidate Rescoring

This appendix records the exact final rescoring formula used in the current implementation. It is separated from the main text because it is a calibration detail rather than the core geometric idea of SFTF. After the coarse and refined candidate sets are built, each candidate n_k is represented by

$$x_k = [J_k, R_k, P_k, B_k, H_k, S_k, \sigma_{1,k}]^T, \quad (33)$$

where H_k is the valid hit count, S_k is the nuclear norm of the face-to-face subtensor, and $\sigma_{1,k}$ is its largest singular value. For each feature $x^{(l)}$, the intra-set rank normalization is

$$\rho_k^{(l)} = \frac{\text{rank}(x_k^{(l)})}{N - 1}, \quad 0 \leq \rho_k^{(l)} \leq 1. \quad (34)$$

The rank of a candidate with a small value is close to 0, and that of a candidate with a large value is close to 1.

The final tuning score is

$$T(n_k) = w_J \rho_k^{(J)} + w_R \rho_k^{(R)} + w_P \rho_k^{(P)} + w_B \rho_k^{(B)} + w_H \rho_k^{(H)} + w_S \rho_k^{(S)} + w_1 \rho_k^{(\sigma_1)}. \quad (35)$$

The in-sample calibration used for Table 1 is

$$\begin{aligned} w_J &= 0, & w_R &= 0.731, & w_P &= -2.3897, \\ w_B &= 2.1447, & w_H &= 2.3109, & w_S &= -2.2922, & w_1 &= 1.6363. \end{aligned} \quad (36)$$

These weights are empirical coefficients learned from comparison with the stored TOMO_INT3 grid. They should be interpreted as one calibrated rescoring example, not as a closed-form physical law. For that reason, the main text also reports Leave-One-Mesh-Out results in §12.3.

C SFTF Candidate vs TOMO_INT3 v_{ss} Landscapes (Group C)

This appendix compares, for each of the five Group C meshes (Bunny, manikin, dragon, happy, Lucy), the SFTF candidate cost landscape against the stored TOMO_INT3 v_{ss} grid used as the comparison baseline in the main text (`*_tomo_int3_vss_grid_1deg_60deg.npz`), to directly show where the SFTF predictions lie on the exhaustive-search landscape. Each grid is the exhaustive-sweep v_{ss} computed at yaw-pitch 1° resolution and a critical angle of 60° , and the entire 361×361 grid is displayed as a filled contour of the RdBu.r diverging colormap (the bluer, the smaller v_{ss} and the better orientation; the redder, the larger support volume; the colormap is the same as the HTML figures in the main text). Crucially, both panels of each figure (Figures 6–10) share the *same* yaw-pitch axes: the left panel scatters the SFTF candidate directions (colored by tuned score), and the right panel shows the TOMO_INT3 v_{ss} landscape, so the SFTF markers occupy identical positions in the two panels. Because a build direction n and its reversal $-n$ describe the same build *axis*, each SFTF direction is placed at the better of its two antipodal grid cells (the smaller- v_{ss} of n and $-n$), i.e. the signed-axis evaluation defined in §10; the markers therefore sit at the same v_{ss} (and ratio) as the main-text tables. Four kinds of markers are used, 3 each, and to avoid confusion we stress that the SFTF markers are SFTF-*score* extrema, not TOMO-verified optima: stars ($\star$, INT3 grid optimal; the three different basins with the smallest v_{ss} on the exhaustive grid), X (INT3 grid worst; the three basins with the largest v_{ss}), circles (SFTF-score best candidates; the three lowest-score SFTF_C++ directions `sftf_cpp_optimal`), and diamonds (SFTF-score worst candidates; the three highest-score directions `sftf_cpp_worst`). The legend is placed below each figure so as not to obscure it. Therefore, the closer the SFTF-score best candidates (circles) lie to the INT3 grid optimal (stars) and the SFTF-score worst (diamonds) to the INT3 grid worst (X), the better the thousands-of-times-cheaper SFTF reproduces the good/bad orientations of the exhaustive search. These landscapes use the TOMO_INT3 exhaustive sweep as the ground-truth reference; the INT3-vs-CUDA *timing* comparison is reported separately in §13 and is not the subject of these figures. These figures were rendered from the stored grid files, the candidate pools, and the verification records supplied with the repository.

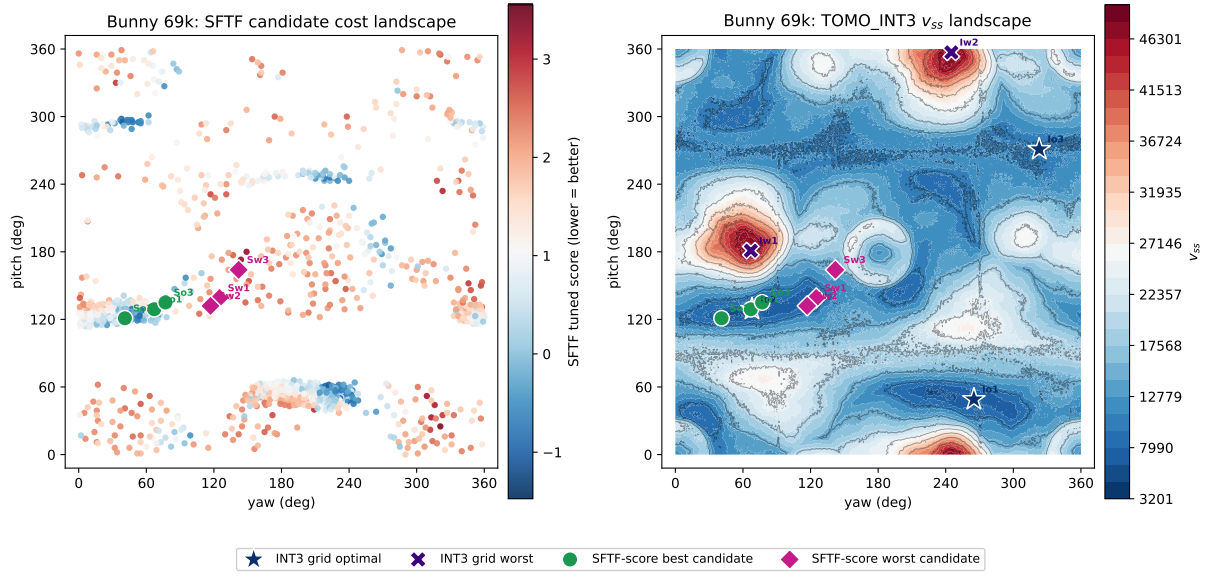


Figure 6: Bunny 69k: SFTF candidate cost landscape (left) and TOMO_INT3 v_{ss} landscape (right), shown on the *same* yaw–pitch axes (blue is good). On the left, each dot is one SFTF candidate direction (placed at its signed-axis TOMO grid cell, §10) colored by the SFTF tuned score; on the right is the exhaustive-sweep v_{ss} landscape. Because both panels share the yaw–pitch coordinates, the SFTF-score best (circles, So) and SFTF-score worst (diamonds, Sw) markers appear at identical positions in the two panels, so one can read off directly how the SFTF prediction lands on the ground-truth landscape. The right panel additionally overlays INT3 grid optimal (stars) and INT3 grid worst (X); markers are 3 each, following the legend below. The global optimum is $v_{ss} = 3201$ (yaw= 265° , pitch= 49° ; Table 1 in the main text), and lies not at a single point but within a broad low-cost basin.

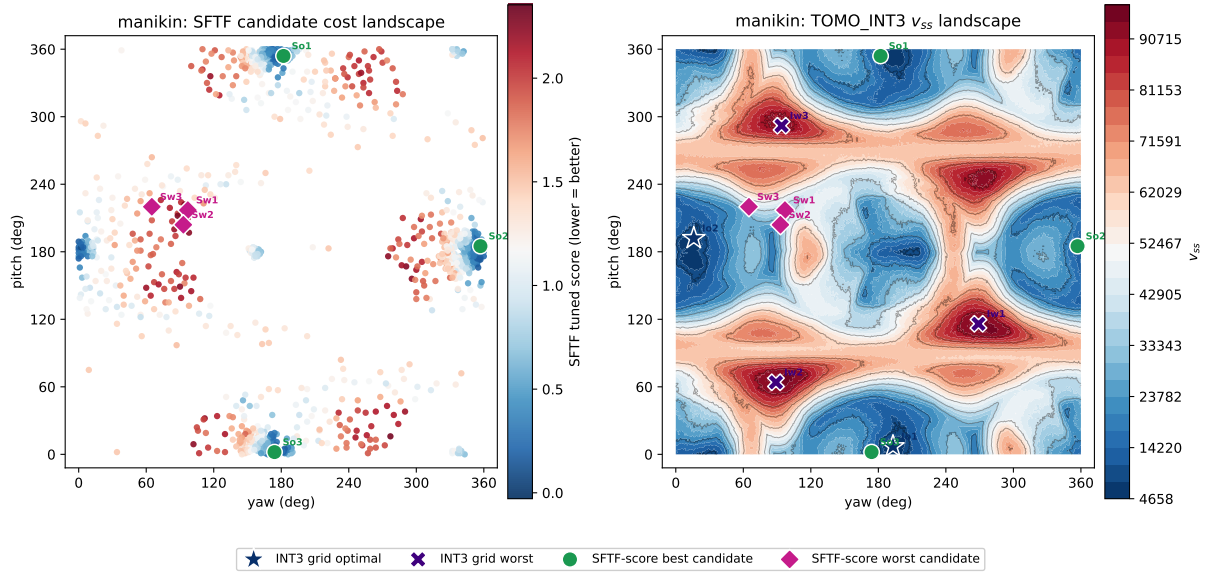


Figure 7: manikin: SFTF candidate cost landscape (left) and TOMO_INT3 v_{ss} landscape (right) on the same yaw–pitch axes (blue is good), with the SFTF-score best (circles, So) and SFTF-score worst (diamonds, Sw) markers at identical positions in both panels and INT3 grid optimal (stars)/worst (X) overlaid on the right. The global optimum is $v_{ss} = 4658$ (yaw= 193° , pitch= 7°); befitting a vertically elongated shape, the optimum sits near a yaw–pitch boundary and the low-cost region is spread over several pitch bands.

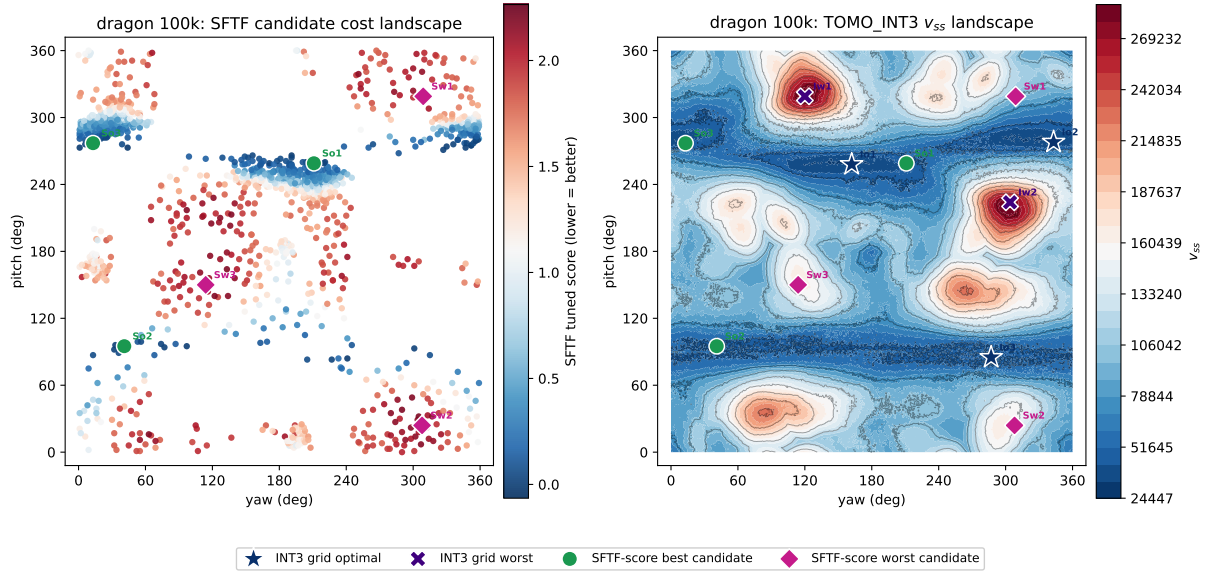


Figure 8: dragon 100k: SFTF candidate cost landscape (left) and TOMO_INT3 v_{ss} landscape (right) on the same yaw–pitch axes (blue is good); the SFTF-score best (circles, So)/worst (diamonds, Sw) markers coincide between panels, with INT3 grid optimal (stars)/worst (X) on the right. The global optimum is $v_{ss} = 24447$ (yaw= 162° , pitch= 258°); the absolute v_{ss} scale is large due to the abundant surface detail, but the global optimum forms a relatively distinct basin.

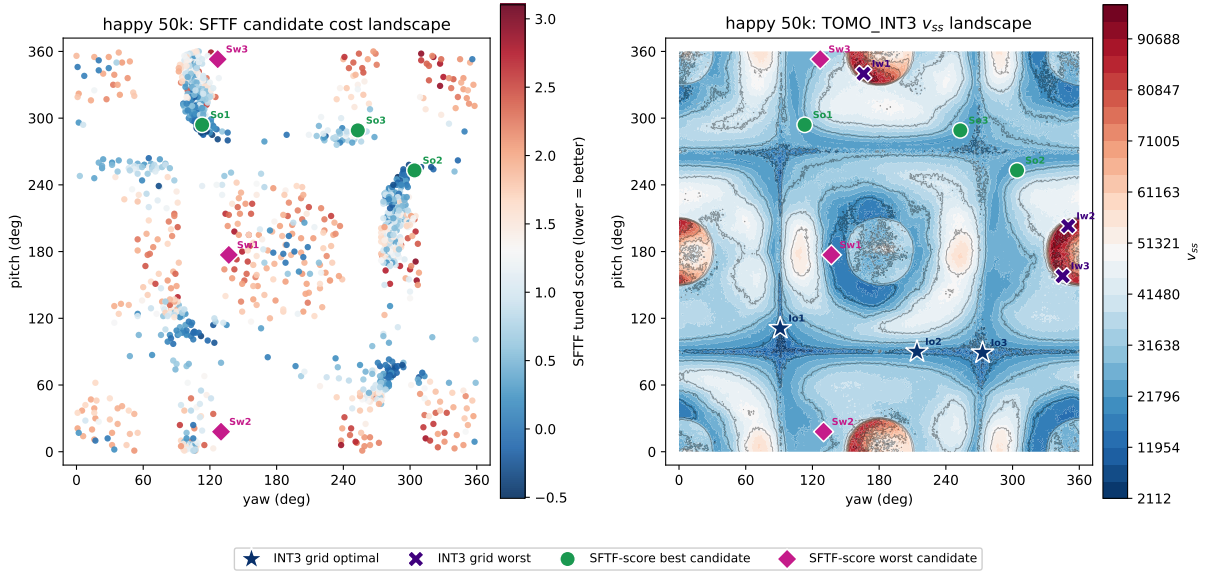


Figure 9: happy 50k: SFTF candidate cost landscape (left) and TOMO_INT3 v_{ss} landscape (right) on the same yaw–pitch axes (blue is good); SFTF-score best (circles, So)/worst (diamonds, Sw) markers coincide between panels, INT3 grid optimal (stars)/worst (X) on the right. The global optimum is $v_{ss} = 2112$ (yaw= 91° , pitch= 111°). As the hard case discussed in §12.5, the low-cost region is narrow and scattered, so the default 512-direction candidate pool does not include a near-optimal orientation, and the SFTF-score best candidates (circles) are also formed at a location away from the INT3 grid optimal (stars) (Table 8).

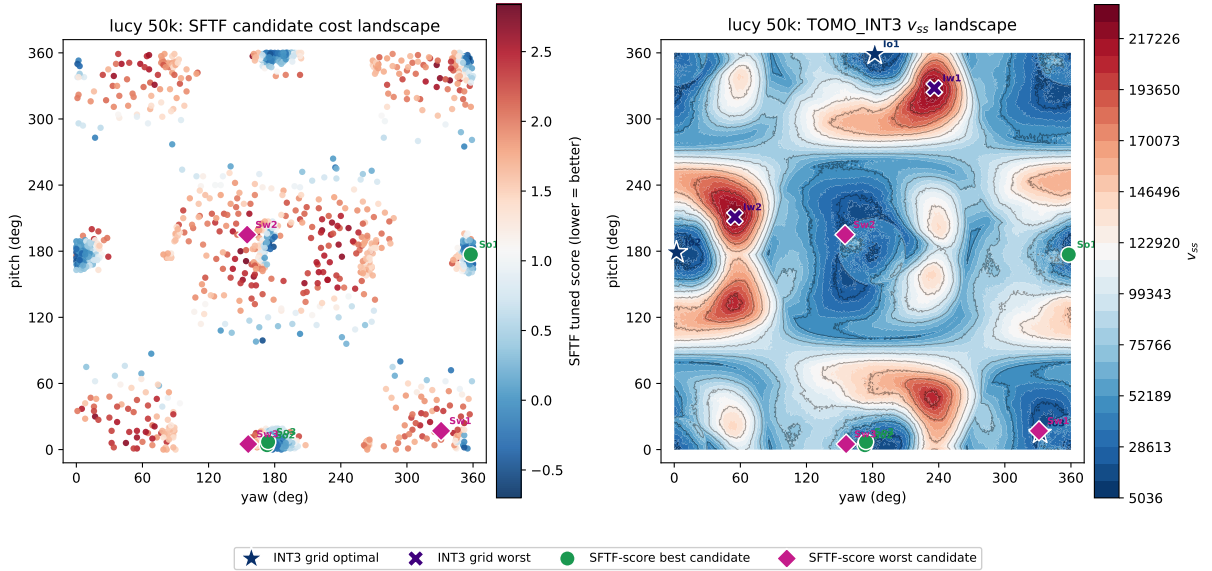


Figure 10: lucy 50k: SFTF candidate cost landscape (left) and TOMO_INT3 v_{ss} landscape (right) on the same yaw–pitch axes (blue is good); SFTF-score best (circles, So)/worst (diamonds, Sw) markers coincide between panels, INT3 grid optimal (stars)/worst (X) on the right. The global optimum is $v_{ss} = 5036$ (yaw= 182° , pitch= 359°); being located near the pitch boundary (359° , drawn with a small margin so boundary markers are not clipped), it requires a basin search that considers the yaw–pitch periodicity.

These combined landscapes visually support the observations of the main text. When the global optimum lies within a broad basin, as in Bunny and dragon, a near-optimal orientation

is easily included in the candidate pool with coarse spherical sampling alone, whereas when the low-cost region is narrow and scattered across several places, as in happy, the sampling resolution becomes a bottleneck. Also, when the optimum is located at a yaw–pitch boundary, as in lacy, it shows the necessity of a basin representation that considers the periodicity emphasized in the main text.

References

- [1] Fan R. K. Chung. *Spectral Graph Theory*. Number 92 in CBMS Regional Conference Series in Mathematics. American Mathematical Society, 1997.
- [2] William G. Cochran. *Sampling Techniques*. Wiley Series in Probability and Statistics. Wiley, New York, 3 edition, 1977.
- [3] Chengkai Dai, Charlie C. L. Wang, Chenming Wu, Sylvain Lefebvre, Guoxin Fang, and Yong-Jin Liu. Support-free volume printing by multi-axis motion. *ACM Transactions on Graphics*, 37(4):1–14, 2018.
- [4] Paramita Das, Kunal Mhapsekar, Souran Chowdhury, Rutuja Samant, and Sam Anand. Selection of build orientation for optimal support structures and minimum part errors in additive manufacturing. *Computer-Aided Design and Applications*, 14(sup1):1–13, 2017.
- [5] Ben Ezair, Fady Massarwi, and Gershon Elber. Orientation analysis of 3D objects toward minimal support volume in 3D-printing. *Computers & Graphics*, 51:117–124, 2015.
- [6] Harald Garcke, Kei Fong Lam, Robert Nürnberg, and Andrea Signori. Overhang penalization in additive manufacturing via phase field structural topology optimization with anisotropic energies. *Applied Mathematics & Optimization*, 87(3):44, 2023.
- [7] Andrew T. Gaynor, Nicholas A. Meisel, Christopher B. Williams, and James K. Guest. Topology optimization for additive manufacturing: Considering maximum overhang constraint. In *15th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, page 2036, 2014.
- [8] Ian Gibson, David Rosen, and Brent Stucker. *Additive Manufacturing Technologies*. Springer, New York, 2 edition, 2015.
- [9] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, 4 edition, 2013.
- [10] Álvaro González. Measurement of areas on a sphere using Fibonacci and latitude–longitude lattices. *Mathematical Geosciences*, 42(1):49–64, 2010.
- [11] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, Waltham, MA, 3 edition, 2011.
- [12] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [13] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, Cambridge, 2 edition, 2013.
- [14] Jingchao Jiang, Jonathan Stringer, Xun Xu, and Ray Y. Zhong. Investigation of printable threshold overhang angle in extrusion-based additive manufacturing for reducing support waste. *International Journal of Computer Integrated Manufacturing*, 31(10):961–969, 2018.

- [15] I. T. Jolliffe. *Principal Component Analysis*. Springer Series in Statistics. Springer, New York, 2 edition, 2002.
- [16] Jae Yeol Jung, Seokjae Chee, and In Hwan Sul. Support structure tomography using per-pixel signed shadow casting in human manikin 3D printing. *Fashion and Textiles*, 9(1):1–18, 2022.
- [17] Ken-Ichi Kanatani. Distribution of directional data and fabric tensors. *International Journal of Engineering Science*, 22(2):149–164, 1984.
- [18] Jae Ryoung Kim and In Hwan Sul. Fast prediction of 3D printing optimal orientation using general-purpose graphic processor unit calculation. *3D Printing and Additive Manufacturing*, 13(1):50–62, 2026.
- [19] Matthijs Langelaar. Combined optimization of part topology, support structure layout and build orientation for additive manufacturing. *Structural and Multidisciplinary Optimization*, 57(5):1985–2004, 2018.
- [20] K. V. Mardia and P. E. Jupp. *Directional Statistics*. Wiley Series in Probability and Statistics. Wiley, Chichester, 2000.
- [21] Amir M. Mirzendehtdel and Krishnan Suresh. Support structure constrained topology optimization for additive manufacturing. *Computer-Aided Design*, 81:1–13, 2016.
- [22] Tomas Möller and Ben Trumbore. Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, 2(1):21–28, 1997.
- [23] Scott D. Roth. Ray casting for modeling solids. *Computer Graphics and Image Processing*, 18(2):109–144, 1982.
- [24] E. B. Saff and A. B. J. Kuijlaars. Distributing many points on a sphere. *The Mathematical Intelligencer*, 19(1):5–11, 1997.
- [25] Seungheon Shin, Yujin Park, and Gil Ho Yoon. Topology optimization for multi-axis additive manufacturing considering overhang and anisotropy. *International Journal of Mechanical Sciences*, 301:110443, 2025.
- [26] Robert Smith and Michael C. Hollenbeck. Build orientation for additive manufacturing of complex structures. U.S. Patent No. 12,343,928, 2025. Assignee: Optisys, Inc.
- [27] Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.
- [28] Mervyn Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–133, 1974.
- [29] Benjamin Vaissier, Jean-Philippe Pernot, Laurent Chougrani, and Philippe Véron. Genetic-algorithm based framework for lattice support structure optimization in additive manufacturing. *Computer-Aided Design*, 110:11–23, 2019.
- [30] Charlie C. L. Wang, Yong-Shun Leung, and Yong Chen. Solid modeling of polyhedral objects by layered depth-normal images on the GPU. *Computer-Aided Design*, 42(6):535–544, 2010.
- [31] Qingnan Zhou and Alec Jacobson. Thingi10k: A dataset of 10,000 3D-printing models. *arXiv preprint arXiv:1605.04797*, 2016.